



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Laurea Magistrale Ingegneria Robotica e
dell'Automazione

Esercizio Identificazione e Stima
Elicottero 2 DoF

Professore:

Prof. Riccardo Costanzi

Candidati:

**Elia Tonci
Luca Cecchi**

ANNO ACCADEMICO 2025/2026

Indice

1	Introduzione	4
1.1	Modello Fisico del Sistema	4
1.2	Modello degli Attuatori e Parametri Fisici	5
1.3	Architettura di Misura e Modello dei Sensori	6
1.3.1	Implementazione del Modello Sensoriale in Simulink	7
2	Scenari di Simulazione e Modellazione dei Disturbi	9
2.1	Scenario 1: Condizioni Nominali	9
2.2	Scenario 2: Disturbo Aerodinamico (Vento)	9
2.3	Architettura Generale in Simulink	10
3	Extended Kalman Filter (EKF)	11
3.1	Implementazione Software e Scelte Progettuali	11
3.2	Risultati della Stima EKF in Condizioni Nominali	14
3.3	Risultati della Stima EKF con Disturbo Aerodinamico	16
4	Unscented Kalman Filter (UKF)	19
4.1	Implementazione e Scelte Progettuali	19
4.2	Risultati della Stima UKF (Scenario Nominale)	22
4.3	Risultati della Stima UKF con Disturbo Aerodinamico	24
4.4	Analisi Quantitativa e Confronto RMSE (EKF vs UKF)	26
5	Smoothing Offline: Algoritmo Rauch-Tung-Striebel (RTS)	28
5.1	Implementazione e Scelte Progettuali	28
5.2	Analisi dei Risultati in Condizioni Nominali (Assenza di Vento) . . .	30
5.3	Analisi dei Risultati con Disturbo Aerodinamico (Vento Continuo) . .	32
6	Sviluppi futuri	36
7	Conclusioni	37

Elenco delle figure

1.1	Schema fisico dell'elicottero 2DOF con indicazione delle forze e dei termini di accoppiamento aerodinamico.	4
1.2	Dettaglio dello schema Simulink del blocco MATLAB Function dei sensori e dell'iniezione del rumore bianco gaussiano.	8
2.1	Architettura generale del sistema in Simulink, configurata per lo Scenario 2 con l'iniezione del vento a $t = 5$ s.	10
3.1	Tracciamento delle traiettorie angolari e delle velocità (EKF vs Ground Truth) in assenza di vento.	15
3.2	Errori EKF a 3-Sigma in assenza di vento.	15
3.3	Innovazione EKF in assenza di vento.	16
3.4	Tracciamento delle traiettorie angolari e delle velocità (EKF vs Ground Truth) con la folata di vento.	17
3.5	Errori EKF a $\pm 3\sigma$ con la folata di vento.	17
3.6	Innovazione EKF con la folata di vento.	18
4.1	Tracciamento delle traiettorie angolari e delle velocità (UKF vs Ground Truth) in assenza di vento.	23
4.2	Errori UKF a $\pm 3\sigma$ in assenza di vento.	23
4.3	Innovazione UKF in assenza di vento.	24
4.4	Tracciamento delle traiettorie angolari e delle velocità (UKF vs Ground Truth) con la folata di vento.	25
4.5	Errori UKF a $\pm 3\sigma$ con la folata di vento.	25
4.6	Innovazione UKF con la folata di vento.	26
4.7	Confronto RMSE in condizioni nominali.	27
4.8	Confronto RMSE con disturbo aerodinamico.	27
5.1	Tracciamento delle traiettorie angolari e delle velocità (EKF vs RTS) in condizioni nominali con mismatch parametrico.	30
5.2	Errore di stima (EKF vs RTS) in assenza di vento.	31
5.3	Incertezza a $\pm 3\sigma$ (EKF vs RTS) in condizioni nominali.	31
5.4	Confronto Errore Quadratico Medio (RMSE) tra EKF e RTS in assenza di vento.	32
5.5	Tracciamento delle traiettorie (EKF vs RTS) in presenza di gradino di vento continuo da $t = 5$ s.	33
5.6	Errore di stima (EKF vs RTS) in presenza di gradino di vento.	33
5.7	Incertezza a $\pm 3\sigma$ (EKF vs RTS) in presenza di vento continuo.	34
5.8	Confronto Errore Quadratico Medio (RMSE) tra EKF e RTS con vento continuo.	34

1. Introduzione

L'obiettivo del presente elaborato è la progettazione, l'analisi e l'implementazione di algoritmi di stima dello stato e filtraggio stocastico per un sistema meccanico non lineare e fortemente accoppiato. Il plant in esame è un modello di Elicottero a due gradi di libertà (2DOF), impiegato per valutare e confrontare le prestazioni di stimatori online — nello specifico l'Extended Kalman Filter (EKF) e l'Unscented Kalman Filter (UKF) — e di un algoritmo di regolarizzazione offline quale lo Smoother di Rauch-Tung-Striebel (RTS). Gli algoritmi operano in uno scenario realistico caratterizzato da marcata incertezza parametrica e da un'architettura di misura asincrona (multi-rate) con letture sensoriali rumorose e indirette.

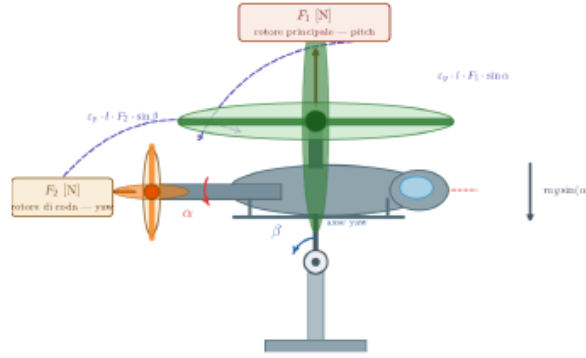


Figura 1.1: Schema fisico dell'elicottero 2DOF con indicazione delle forze e dei termini di accoppiamento aerodinamico.

Il sistema ruota attorno a un perno fisso a terra: l'angolo di pitch α descrive la rotazione attorno all'asse orizzontale, controllata dalla forza generata dal rotore principale $u_1 = F_1$, mentre l'angolo di yaw β descrive la rotazione attorno all'asse verticale, controllata dalla forza del rotore di coda $u_2 = F_2$.

1.1 Modello Fisico del Sistema

Per simulare coerentemente la dinamica dell'elicottero, il plant fisico è stato modellato definendo come variabili di stato gli angoli e le velocità angolari dei due assi. Il vettore di stato del sistema è quindi definito come:

$$x = [\alpha \quad \dot{\alpha} \quad \beta \quad \dot{\beta}]^T = [x_1 \quad x_2 \quad x_3 \quad x_4]^T \quad (1.1)$$

mentre il vettore degli ingressi è $u = [u_1 \quad u_2]^T = [F_1 \quad F_2]^T$.

La non-linearità del sistema è introdotta principalmente dall'inerzia di yaw, J_β , la quale non è costante ma varia in funzione dell'assetto di pitch (x_1), secondo la

relazione:

$$J_\beta(x_1) = J_y \sin^2(x_1) + (J_z + ml^2) \cos^2(x_1) + I_b \quad (1.2)$$

Inoltre, il sistema presenta un marcato accoppiamento sia geometrico che aerodinamico tra i due assi. La coppia di pitch dipende dall'orientamento yaw tramite $lF_1 \cos \beta$, e viceversa, quella di yaw dipende dall'assetto di pitch tramite $lF_2 \cos \alpha$. Sono presenti anche effetti di *cross-thrust* aerodinamico, modellati dai coefficienti ϵ_p ed ϵ_y .

Rispetto alle modellazioni semplificate, in questa trattazione l'interazione aerodinamica con l'ambiente è ricavata proiettando il vettore di velocità del vento esterno nel Body frame dell'elicottero tramite la matrice di rotazione $R_{I \rightarrow B}$ (definita per Roll nullo):

$$R_{I \rightarrow B}(\alpha, \beta) = \begin{bmatrix} \cos \alpha \cos \beta & \cos \alpha \sin \beta & -\sin \alpha \\ -\sin \beta & \cos \beta & 0 \\ \sin \alpha \cos \beta & \sin \alpha \sin \beta & \cos \alpha \end{bmatrix} \quad (1.3)$$

La velocità relativa dell'aria rispetto ai rotori (alloggiati a distanza l sull'asse X_B) è espressa come:

$$v_{rel,B} = R_{I \rightarrow B} v_{wind,I} - (\omega_B \times r_{cp}) \quad (1.4)$$

dove $\omega_B = [-\dot{\beta} \sin \alpha \quad \dot{\alpha} \quad \dot{\beta} \cos \alpha]^T$ è la velocità angolare in Body e $r_{cp} = [l \quad 0 \quad 0]^T$. La forza di drag risultante $F_{drag,B} = \frac{1}{2} \rho (C_d A) \odot v_{rel,B} \|v_{rel,B}\|$ introduce le coppie di disturbo aerodinamico:

$$\tau_{wind,\alpha} = -F_{drag,B}(3) \cdot l, \quad \tau_{wind,\beta} = F_{drag,B}(2) \cdot l \quad (1.5)$$

Le equazioni differenziali del moto utilizzate per la simulazione del plant reale sono espresse nella seguente forma nello spazio di stato:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = \frac{1}{J_\alpha} \left(-c_\alpha x_2 - mgl \sin(x_1) + lu_1 \cos(x_3) + \epsilon_p lu_2 \sin(x_3) + \tau_{wind,\alpha} \right) \\ \dot{x}_3 = x_4 \\ \dot{x}_4 = \frac{1}{J_\beta(x_1)} \left(-c_\beta x_4 + lu_2 \cos(x_1) + \epsilon_y lu_1 \sin(x_1) + \tau_{wind,\beta} \right) \end{cases} \quad (1.6)$$

1.2 Modello degli Attuatori e Parametri Fisici

Per una modellazione più realistica, le dinamiche dei rotori sono state approssimate con funzioni di trasferimento del secondo ordine, includendo un ritardo di trasporto $\bar{\tau}_d$ (approssimato tramite Padé del primo ordine):

$$\bar{G}_m(s) = \frac{\bar{\omega}_n^2}{s^2 + 2\bar{\zeta}\bar{\omega}_n s + \bar{\omega}_n^2} e^{-\bar{\tau}_d s} \quad (1.7)$$

Per valutare la robustezza e la capacità di convergenza dei filtri in condizioni di incertezza sul modello, è stato introdotto un *mismatch parametrico* intenzionale del $\pm 15\%$ tra i parametri reali del plant (p) e quelli impostati all'interno degli algoritmi di stima ($\hat{p}$). Nel dettaglio, i parametri adottati per la simulazione dell'elicottero e quelli noti ai filtri sono riportati nella Tabella 1.1 [7].

Parametro	Descrizione	Valore reale (p)	Valore stimatore ($\hat{p}$)
J_α	Inerzia di pitch	0.012 kg m ²	0.0138 kg m ²
J_y	Componente inerziale asse y	0.00023 kg m ²	0.0002645 kg m ²
J_z	Componente inerziale asse z	0.00364 kg m ²	0.003094 kg m ²
I_b	Inerzia di base yaw	0.00023 kg m ²	0.0001955 kg m ²
m	Massa dell'elicottero	0.2 kg	0.2 kg
l	Distanza del baricentro/rotori	0.2 m	0.2 m
c_α	Smorzamento asse pitch	0.01 N m s/rad	0.0115 N m s/rad
c_β	Smorzamento asse yaw	0.01 N m s/rad	0.0085 N m s/rad
ϵ_p	Coefficiente cross-thrust (pitch)	0.1	0.115
ϵ_y	Coefficiente cross-thrust (yaw)	0.1	0.085
Cd_A	Coefficienti drag equivalenti $[X; Y; Z]$	$[0.1; 0.1; 0.2]$ m ²	$[0.11; 0.09; 0.22]$ m ²
$\bar{\omega}_n$	Pulsazione naturale attuatori	30 rad/s	30 rad/s
$\bar{\zeta}$	Smorzamento attuatori	0.7	0.7
$\bar{\tau}_d$	Ritardo di trasporto	20 ms	20 ms

Tabella 1.1: Parametri fisici reali del modello Elicottero 2DOF e relativi parametri affetti da incertezza utilizzati negli stimatori.

1.3 Architettura di Misura e Modello dei Sensori

Per la stima in tempo reale dello stato del sistema, si è optato per l'adozione di una singola unità di misura inerziale (IMU) commerciale integrata, nello specifico il modello *InvenSense MPU-9250*. Sebbene alloggiata su un unico chip, tale architettura ospita al suo interno due trasduttori fisicamente distinti e di natura differente, soddisfacendo pienamente i requisiti di eterogeneità sensoriale richiesti. Il dispositivo racchiude infatti un accelerometro triassiale MEMS e un magnetometro triassiale *Asahi Kasei AK8963*.

Al fine di rendere la simulazione numerica aderente alla realtà sperimentale, l'architettura opera in configurazione *multi-rate*:

- L'accelerometro lavora ad alta frequenza ed è campionato in modo continuo al passo fondamentale della simulazione (1000 Hz, ovvero $dt = 0.001$ s).
- Il magnetometro lavora a una frequenza ridotta pari a 100 Hz ($T_{c_{mag}} = 0.01$ s). Negli istanti intermedi in cui la misura non è pronta, il blocco sensori genera un valore non disponibile, mascherato tramite un flag logico NaN (Not a Number).

Il vettore di misura completo ha dimensione 3×1 , definito come $y_k = [y_{acc} \quad m_x \quad m_y]^T$, dove le componenti sfruttano la lettura indiretta delle variabili di assetto descritte dalle seguenti relazioni geometriche non lineari:

- **Accelerometro (Inclinometro):** Sfruttato lungo l'asse X solidale al corpo dell'elicottero, misura la proiezione dell'accelerazione di gravità lungo il proprio asse. Assumendo dinamiche transitorie lente o condizioni di quasi-equilibrio, il sensore opera come inclinometro per ricavare l'angolo di pitch α secondo il modello:

$$y_{acc} = -g \sin(\alpha) + v_1 \quad (1.8)$$

dove $v_1 \sim \mathcal{N}(0, \sigma_{v,acc}^2)$ rappresenta il rumore bianco gaussiano additivo ($\sigma_{v,acc} = 0.25$).

- **Magnetometro (Bussola digitale sdoppiata):** Rileva le variazioni del campo magnetico terrestre lungo gli assi X e Y del Body frame per migliorare l'osservabilità dello yaw β anche in presenza di repentine variazioni di pitch:

$$\begin{cases} m_x = \cos(\alpha) \cos(\beta) + v_2 \\ m_y = -\sin(\beta) + v_3 \end{cases} \quad (1.9)$$

dove $v_2, v_3 \sim \mathcal{N}(0, \sigma_{v,mag}^2)$ descrivono i rumori bianchi gaussiani indipendenti dei due canali magnetometrici ($\sigma_{v,mag} = 0.1$).

1.3.1 Implementazione del Modello Sensoriale in Simulink

Dal punto di vista dell'architettura di simulazione in ambiente Simulink, il modello dei sensori è stato centralizzato all'interno di un blocco *MATLAB Function* denominato `sensori_imu`, che riceve in ingresso lo stato del sistema x , la struttura dei parametri p e il tempo di simulazione t .

Il codice MATLAB implementato all'interno del blocco embedded è riportato nel Listing 1.1.

```

1 function y_out = sensori_imu(x, p, t)
2     % Input: Vettore di stato x, parametri p, tempo di simulazione t
3     % Output: Vettore delle misure y_out = [y_acc; mx; my]
4
5     % Estrazione degli angoli di assetto
6     alpha = x(1); % Pitch [rad]
7     beta  = x(3); % Yaw [rad]
8
9     % EQUAZIONI DEI SENSORI (Misure ideali)
10    % 1. Accelerometro asse X (misura per alpha)
11    y_acc = -p.g * sin(alpha);
12
13    % 2. Magnetometro asse X e Y (misura per beta)
14    mx = cos(alpha)*cos(beta);
15    my = -sin(beta);
16
17    y_out = [y_acc; mx; my];
18
19    % =====

```

```

20 % LOGICA MULTI-RATE (Mascheramento dati mancanti)
21 % =====
22 Tc_mag = 0.01; % Tempo di campionamento del sensore lento
    [s]
23 tol = 1e-4; % Tolleranza per arrotondamenti numerici di
    Simulink
24
25 % Calcolo il resto della divisione tra il tempo attuale e
    Tc_mag
26 resto = mod(t, Tc_mag);
27
28 % Se il tempo attuale non e' un multiplo esatto del passo
    del magnetometro
29 if resto > tol && (Tc_mag - resto) > tol
30     % Il magnetometro non ha un nuovo dato: maschero le
        sue uscite
31     y_out(2) = NaN;
32     y_out(3) = NaN;
33 end
34 end

```

Listing 1.1: Codice della MATLAB Function per il calcolo delle misure in configurazione asincrona multi-rate.

Durante gli istanti di campionamento in cui le misure del magnetometro non sono disponibili (ovvero impostate a NaN), gli stimatori online EKF e UKF attivano una logica di correzione parziale. Tramite un controllo condizionale (`isnan(y_k)`), i filtri escludono temporaneamente le righe e le colonne associate ai sensori inattivi dal vettore di misura y_k , dalla misura attesa $\hat{y}$ e dalle matrici Jacobiane di misura H (o dalle covarianze incrociate nell'UKF), effettuando l'aggiornamento dello stato unicamente sulla base della lettura accelerometrica attiva.

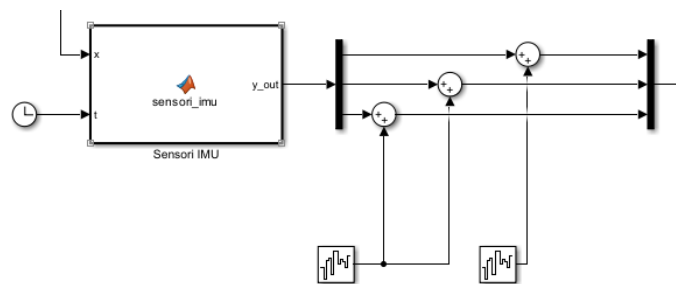


Figura 1.2: Dettaglio dello schema Simulink del blocco MATLAB Function dei sensori e dell'iniezione del rumore bianco gaussiano.

2. Scenari di Simulazione e Modellazione dei Disturbi

Per valutare in modo completo e rigoroso le prestazioni degli stimatori (EKF e UKF), il piano di validazione è stato articolato su due scenari di test sequenziali. Poiché l'obiettivo degli osservatori è stimare lo stato del plant reale, il codice interno dei filtri e le relative matrici di covarianza (Q ed R) rimangono assolutamente invariati tra le simulazioni. La finalità è verificare la capacità degli algoritmi di adattarsi al mismatch parametrico illustrato nel capitolo precedente e di reiettare disturbi esogeni non modellati matematicamente al loro interno.

2.1 Scenario 1: Condizioni Nominali

Il primo test è finalizzato a validare l'implementazione matematica dei filtri e l'accuratezza dei Jacobiani analitici esatti ricavati offline. In questo scenario, l'impianto fisico virtuale opera in assenza di disturbi ambientali esterni. Il sistema viene sollecitato esclusivamente dagli ingressi di controllo (u_1 e u_2), generati come segnali sinusoidali sovrapposti per garantire la persistenza dell'eccitazione e sollecitare le dinamiche transitorie senza portare l'elicottero fuori dal regime dei piccoli angoli.

2.2 Scenario 2: Disturbo Aerodinamico (Vento)

Al fine di valutare la robustezza degli stimatori in condizioni operative estreme, il secondo scenario introduce un disturbo esogeno severo: il vento. La presenza di perturbazioni atmosferiche mette rigorosamente alla prova la capacità dei filtri di mantenere una stima coerente, specialmente se sommate agli errori di modellazione parametrica già affrontati dall'osservatore.

In ambiente Simulink, il profilo del vento è stato generato introducendo un vettore di velocità nel sistema di riferimento inerziale. Il disturbo è stato modellato tramite una funzione a gradino impostata per attivarsi all'istante $t = 5$ s, iniettando una folata continua con intensità pari a 1 m/s che si mantiene costante fino al termine della simulazione.

A differenza di approcci semplificati in cui il disturbo viene sommato a posteriori, l'interazione aerodinamica è stata modellata fisicamente. La velocità del vento viene proiettata nel *Body frame* dell'elicottero per calcolare la velocità relativa dell'aria. Sfruttando la densità dell'aria e il parametro di resistenza equivalente reale dell'impianto ($C_d A$), viene calcolato il vettore delle forze di drag, il quale genera coppie di disturbo attorno agli assi di beccheggio e imbardata. Tali coppie vengono sommate ai momenti di controllo dei rotori, alterando le accelerazioni angolari risultanti ($\ddot{\alpha}$ e $\ddot{\beta}$).

2.3 Architettura Generale in Simulink

L'integrazione della dinamica nominale, del disturbo e dei modelli sensoriali è stata centralizzata in un'architettura Simulink completa, illustrata in Figura 2.1.

La sezione di sinistra rappresenta il *Ground Truth* (l'impianto fisico reale), sollecitato dai comandi armonici e, quando previsto, dal gradino del vento. L'impianto produce le variabili di stato vere che vengono passate al blocco dei sensori IMU. Quest'ultimo, operando in logica *multi-rate*, provvede a inquinare i dati con rumore bianco gaussiano additivo e a mascherare le misurazioni del magnetometro con valori NaN negli istanti in cui il sensore non ha ancora prodotto un nuovo campionamento, ricavando così il vettore di misura finale $y_k \in \mathbb{R}^3$.

La sezione di destra ospita i due stimatori embedded, EKF e UKF. Questi operano a ciclo chiuso utilizzando ritardi unitari ($1/z$) per propagare ricorsivamente la stima dello stato e della covarianza all'istante di campionamento successivo. Grazie alla logica di isolamento dei sensori attivi e all'analisi dell'innovazione tramite la distanza di Mahalanobis (*Gating*), i filtri sono in grado di gestire dinamicamente i dati mancanti e rigettare eventuali *outlier*.

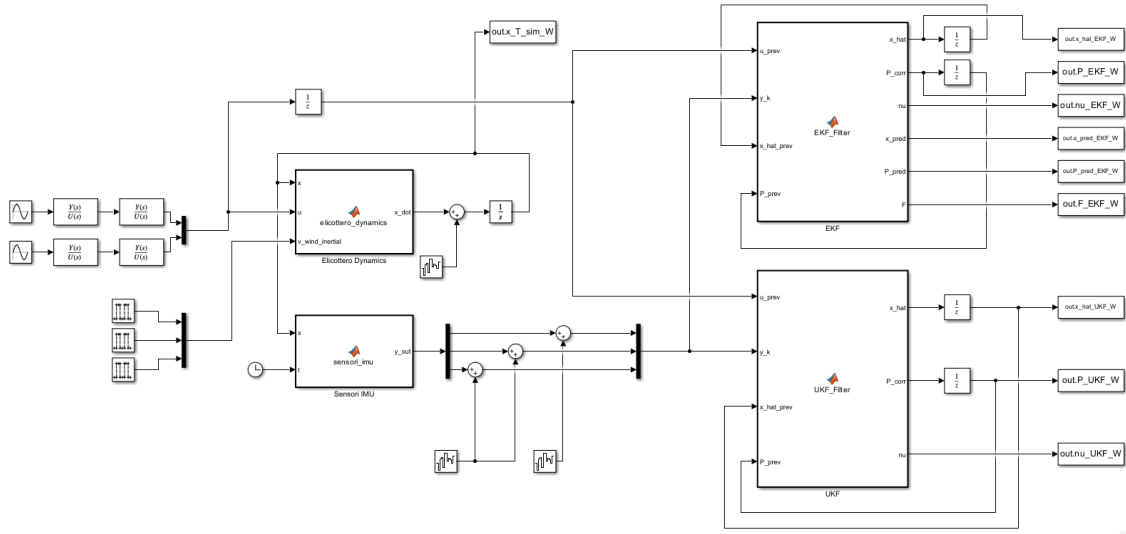


Figura 2.1: Architettura generale del sistema in Simulink, configurata per lo Scenario 2 con l'iniezione del vento a $t = 5$ s.

3. Extended Kalman Filter (EKF)

L'Extended Kalman Filter (EKF) rappresenta lo standard industriale per l'estensione del Filtro di Kalman classico ai sistemi non lineari [6]. Il suo principio di funzionamento si basa sulla linearizzazione locale delle equazioni della dinamica e di misura attorno alla stima dello stato corrente. L'algoritmo si articola in due fasi ricorsive ad ogni passo di campionamento:

1. **Predizione (a priori):** Lo stato viene propagato in avanti nel tempo utilizzando il modello matematico non lineare del sistema, mentre la matrice di covarianza dell'errore viene aggiornata sfruttando la matrice Jacobiana di transizione dello stato (F), che rappresenta l'approssimazione lineare della dinamica in quel preciso istante.
2. **Correzione (a posteriori):** La stima a priori viene corretta confrontando le uscite previste dal modello con le misurazioni effettive dei sensori, mediate e pesate tramite il Guadagno di Kalman.

3.1 Implementazione Software e Scelte Progettuali

L'algoritmo EKF è stato integrato nell'architettura Simulink avvalendosi di un blocco MATLAB Function operante a ciclo discreto con un passo di integrazione ad alta frequenza $dt = 0.001$ s (1000 Hz). Per gestire l'elevata complessità del *plant* e garantire la solidità numerica in presenza dell'architettura sensoriale multi-rate, sono state adottate soluzioni progettuali specifiche:

- **Jacobiani Analitici Offline:** A differenza degli approcci basati su linearizzazione numerica (tramite perturbazioni infinitesime locali) che possono introdurre oneri computazionali e instabilità, le matrici Jacobiane della dinamica (F) e di misura (H) sono state ricavate analiticamente in modo esatto. Avvalendosi del *Symbolic Math Toolbox* di MATLAB, le derivate parziali del modello — comprensive dell'inerzia variabile di yaw e degli effetti aerodinamici — sono state pre-calcolate e compilate in una funzione esterna (`jacobiani_analitici.m`), alleggerendo l'esecuzione *real-time*.
- **Gestione Multi-Rate e Gating Statistico:** Poiché il magnetometro opera a una frequenza inferiore rispetto all'accelerometro (restituendo vettori NaN tra un campionamento e l'altro), la fase di correzione è stata resa dinamica. Il filtro "ritaglia" vettori e matrici considerando solo i sensori attivi. Inoltre, è stato implementato un meccanismo di rigetto degli *outlier* basato sulla Distanza di Mahalanobis: l'innovazione viene validata tramite un test del Chi-Quadro (con soglie a 3.84 nel caso della sola misura dell'accelerometro e 7.81 quando risultano disponibili tutte e tre le misure). Se la distanza supera la soglia, la

misura viene ignorata e il filtro si affida unicamente alla predizione, garantendo estrema robustezza.

Infine, il blocco riceve in ingresso i comandi di controllo ritardati di un passo di integrazione (u_{k-1}), le misure attuali (y_k), lo stato stimato al passo precedente ($\hat{x}_{k-1}$) e la struttura dei parametri perturbati ($\hat{p}$), riflettendo l'incertezza sul modello. Nel Listing 3.1 viene riportata l'implementazione completa utilizzata per la simulazione.

```

1 function [x_hat, P_corr, nu, x_pred, P_pred, F] =
   EKF_Filter(u_prev, y_k, x_hat_prev, P_prev, Q, R, dt,
   p_hat)
2 % =====
3 % 1. FASE DI PREDIZIONE
4 % =====
5 alpha = x_hat_prev(1); alpha_dot = x_hat_prev(2);
6 beta = x_hat_prev(3); beta_dot = x_hat_prev(4);
7 F1 = u_prev(1); F2 = u_prev(2);
8 rho = 1.225; % Densita' dell'aria [kg/m^3]
9 Cd_A = [0.11; 0.09; 0.22]; % Prodotto Cd * Area
   perturbato [X; Y; Z]
10
11 J_beta = p_hat.J_y * sin(alpha)^2 + (p_hat.J_z +
   p_hat.m*p_hat.l^2)*cos(alpha)^2 + p_hat.I_b;
12 omega_body = [-beta_dot * sin(alpha); alpha_dot; beta_dot
   * cos(alpha)];
13 v_corpo_body = cross(omega_body, [p_hat.l; 0; 0]);
14 v_rel_body = -v_corpo_body; % Il filtro assume vento
   esterno nullo
15 v_rel_norm = norm(v_rel_body);
16
17 F_drag_body = 0.5 * rho * Cd_A .* v_rel_body * v_rel_norm;
18 tau_drag_alpha = -F_drag_body(3) * p_hat.l;
19 tau_drag_beta = F_drag_body(2) * p_hat.l;
20 alpha_ddot = (-p_hat.c_alpha*alpha_dot -
   p_hat.m*p_hat.g*p_hat.l*sin(alpha) +
   p_hat.l*F1*cos(beta) +
   p_hat.eps_p*p_hat.l*F2*sin(beta) + tau_drag_alpha) /
   p_hat.J_alpha;
21 beta_ddot = (-p_hat.c_beta*beta_dot +
   p_hat.l*F2*cos(alpha) +
   p_hat.eps_y*p_hat.l*F1*sin(alpha) + tau_drag_beta) /
   J_beta;
22
23 x_pred = [alpha + dt * alpha_dot;
24           alpha_dot + dt * alpha_ddot;
25           beta + dt * beta_dot;
26           beta_dot + dt * beta_ddot];
27
28 % Jacobiano analitico F
29 [F, ~] = jacobiani_analitici(x_hat_prev(1),
   x_hat_prev(2), x_hat_prev(3), x_hat_prev(4), ...

```

```

30         u_prev(1), u_prev(2), dt, rho, ...
31         p_hat.m, p_hat.l, p_hat.g,
32         p_hat.J_y, p_hat.J_z,
33         p_hat.I_b, p_hat.J_alpha, ...
34         p_hat.c_alpha, p_hat.c_beta,
35         p_hat.eps_p, p_hat.eps_y, ...
36         Cd_A(1), Cd_A(2), Cd_A(3));
37
38     P_pred = F * P_prev * F' + Q;
39
40     % =====
41     % 2. FASE DI CORREZIONE
42     % =====
43     hat_y = [-p_hat.g * sin(x_pred(1));
44             cos(x_pred(1)) * cos(x_pred(3));
45             -sin(x_pred(3))];
46
47     % Jacobiano analitico H completo
48     [~, H] = jacobiani_analitici(x_pred(1), x_pred(2),
49     x_pred(3), x_pred(4), ...
50     u_prev(1), u_prev(2), dt, rho, ...
51     p_hat.m, p_hat.l, p_hat.g,
52     p_hat.J_y, p_hat.J_z,
53     p_hat.I_b, p_hat.J_alpha, ...
54     p_hat.c_alpha, p_hat.c_beta,
55     p_hat.eps_p, p_hat.eps_y, ...
56     Cd_A(1), Cd_A(2), Cd_A(3));
57
58     % Identificazione sensori validi (Multi-Rate)
59     idx_validi = ~isnan(y_k);
60
61     x_hat = x_pred;
62     P_corr = P_pred;
63     nu = [0; 0; 0];
64
65     if any(idx_validi)
66         y_k_valid = y_k(idx_validi);
67         hat_y_valid = hat_y(idx_validi);
68         H_valid = H(idx_validi, :);
69         R_valid = R(idx_validi, idx_validi);
70
71         nu_valid = y_k_valid - hat_y_valid;
72         S = H_valid * P_pred * H_valid' + R_valid;
73
74         % DISTANZA DI MAHALANOBIS (Gating)
75         D2 = nu_valid' * (S \ nu_valid);
76
77         gradi_liberta = sum(idx_validi);
78         if gradi_liberta == 1
79             soglia = 3.84;
80         else
81             soglia = 7.81;
82     end

```

```

75         end
76
77         if D2 < soglia
78             L = P_pred * H_valid' / S;
79             x_hat = x_pred + L * nu_valid;
80             P_corr = (eye(4) - L * H_valid) * P_pred;
81             nu(idx_validi) = nu_valid;
82         end
83     end
84 end

```

Listing 3.1: Codice della funzione MATLAB per l'Extended Kalman Filter con logica Multi-Rate e Jacobiani analitici.

3.2 Risultati della Stima EKF in Condizioni Nominali

Per validare l'efficacia del Filtro di Kalman Esteso, il sistema è stato inizialmente simulato in condizioni nominali (in assenza di vento), ma sottoponendo lo stimatore al mismatch parametrico illustrato in precedenza ($\pm 15\%$ sulle dinamiche inerziali e di smorzamento). I segnali di controllo sinusoidali iniettati mantengono l'elicottero nel regime dei "piccoli angoli".

In questo scenario operativo, la stima prodotta dall'EKF risulta molto accurata. Questa rapida convergenza, pur in presenza di parametri imprecisi, è giustificata da tre fattori tecnici chiave:

- **Quasi-Linearità:** Date le modeste ampiezze di oscillazione, le forti non-linearità del sistema (come la dipendenza del momento d'inerzia dal $\sin^2(\alpha)$) non si manifestano in modo aggressivo, rendendo l'approssimazione al primo ordine dell'EKF accurata.
- **Campionamento ad Altissima Frequenza:** L'esecuzione a 1000 Hz ($dt = 0.001$ s) garantisce che l'errore di linearizzazione e l'incertezza dovuta ai parametri errati non abbiano il tempo materiale di crescere e causare derive numeriche prima dell'arrivo della misura accelerometrica successiva.
- **Jacobiano ricavato analiticamente:** L'impiego delle espressioni simboliche pre-calcolate fornisce una matrice di transizione di stato matematicamente coerente in ogni istante, massimizzando l'efficacia del tuning effettuato sulla matrice Q .

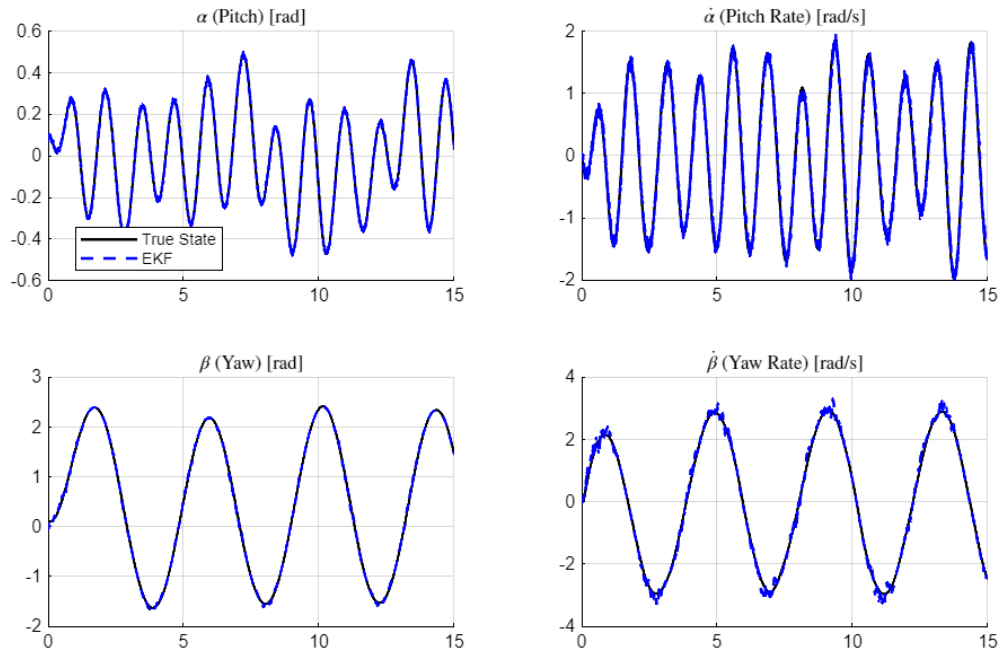


Figura 3.1: Tracciamento delle traiettorie angolari e delle velocità (EKF vs Ground Truth) in assenza di vento.

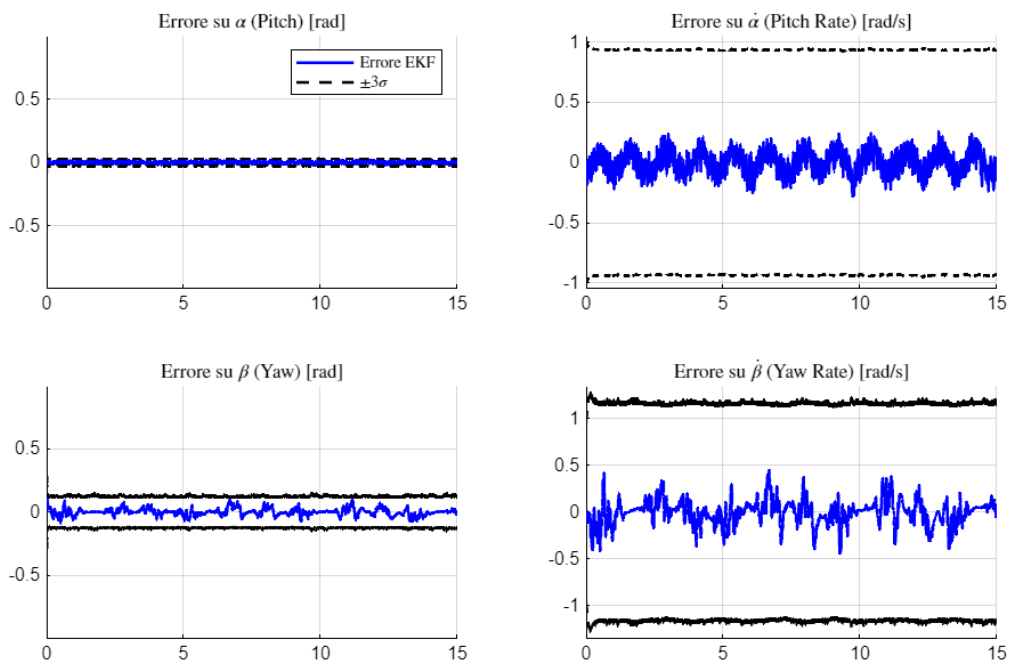


Figura 3.2: Errori EKF a 3-Sigma in assenza di vento.

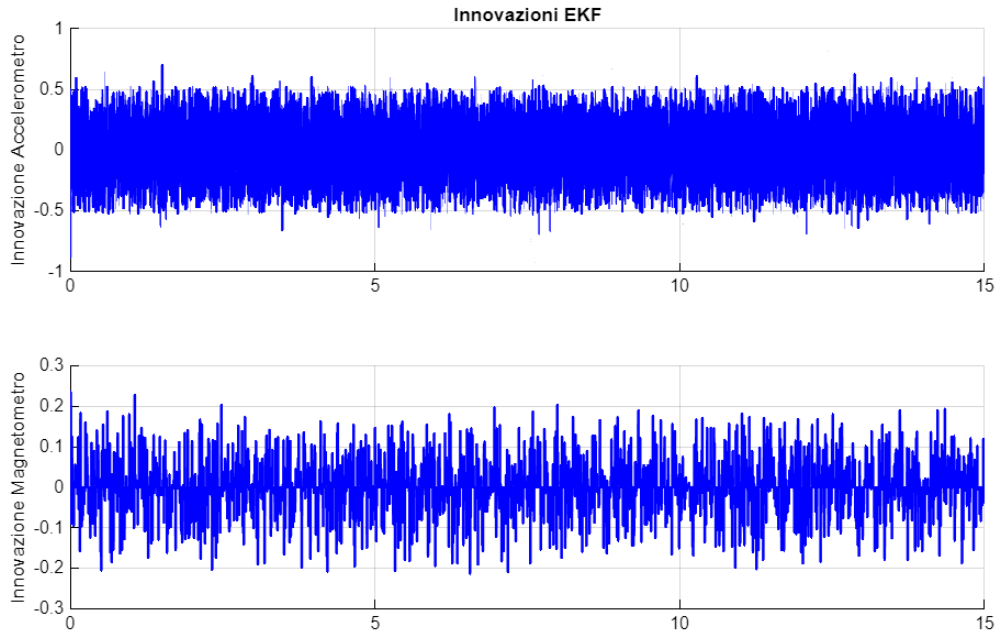


Figura 3.3: Innovazione EKF in assenza di vento.

L'analisi di consistenza statistica conferma l'eccellente validità del tuning effettuato sulle matrici Q ed R , le quali riescono ad assorbire egregiamente l'incertezza di modello introdotta tramite $\hat{p}$. Gli errori di stima presentano media rigorosamente nulla e si mantengono sempre all'interno dei limiti teorici di confidenza a $\pm 3\sigma$. L'innovazione del filtro — ovvero il residuo tra la misura reale catturata dai sensori e quella predetta dal modello — presenta un andamento stazionario, confermando che l'algoritmo sta estraendo tutta l'informazione utile senza subire derive. Abbiamo osservato, il passaggio positivo del test di Anderson ci dice che l'innovazione risulta essere un rumore bianco per i sensori, indicando un corretto tuning dello stimatore.

3.3 Risultati della Stima EKF con Disturbo Aerodinamico

Il banco di prova più severo per valutare le prestazioni dell'EKF è l'introduzione di un gradino continuo di vento a $t = 5$ s, combinato al preesistente mismatch parametrico. Come visibile nel codice dell'algoritmo (Listing 3.1, Riga 14), il modello interno del filtro assume esplicitamente l'assenza di vento esterno ($\mathbf{v_rel_body} = -\mathbf{v_corpo_body}$). Di conseguenza, il filtro affronta un disturbo prolungato e totalmente incognito.

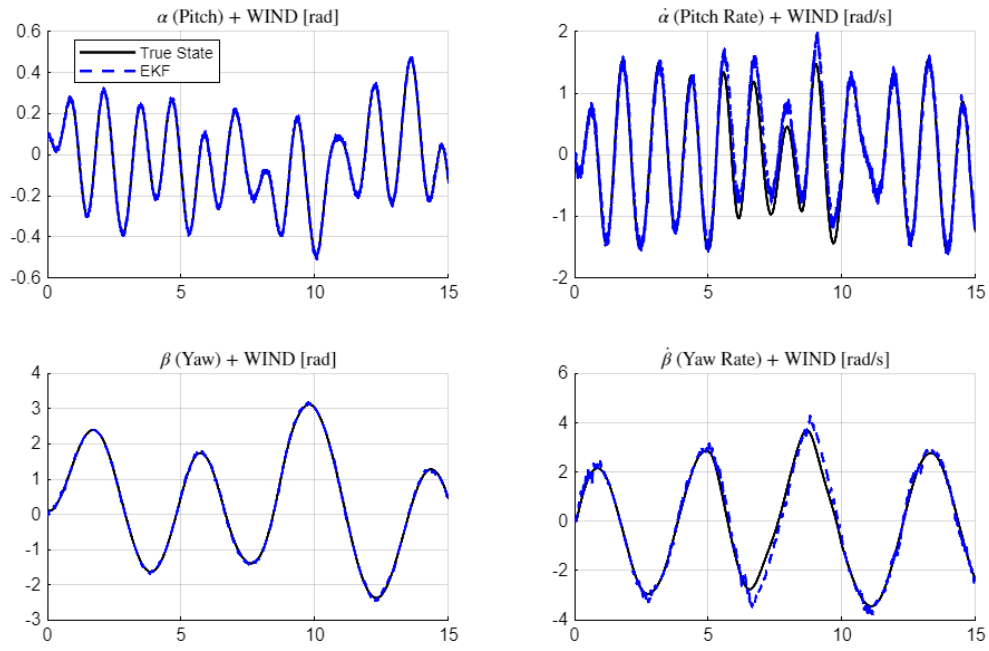


Figura 3.4: Tracciamento delle traiettorie angolari e delle velocità (EKF vs Ground Truth) con la folata di vento.

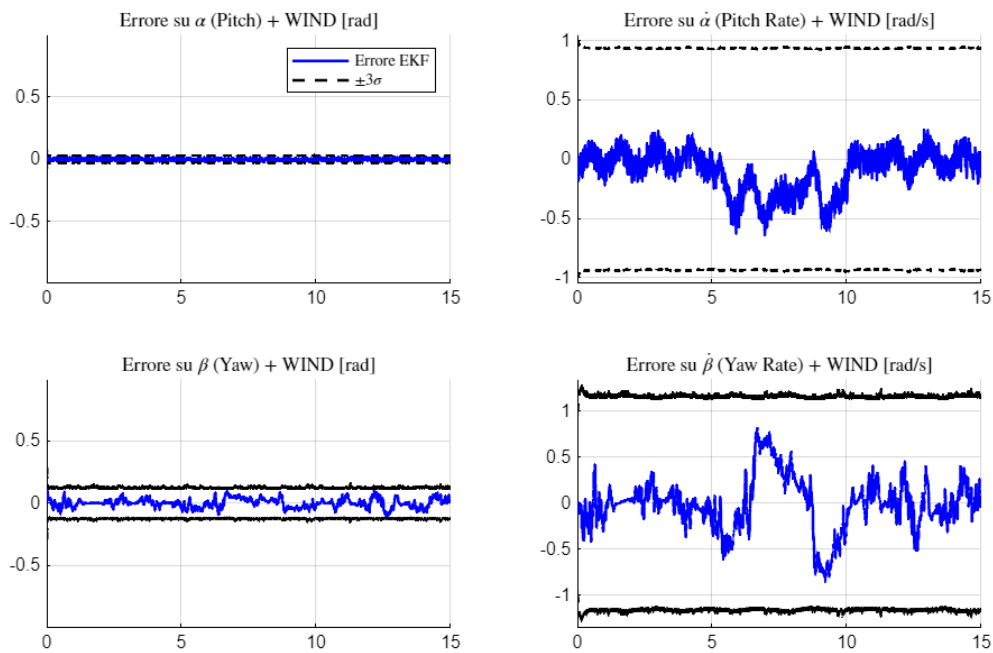


Figura 3.5: Errori EKF a $\pm 3\sigma$ con la folata di vento.

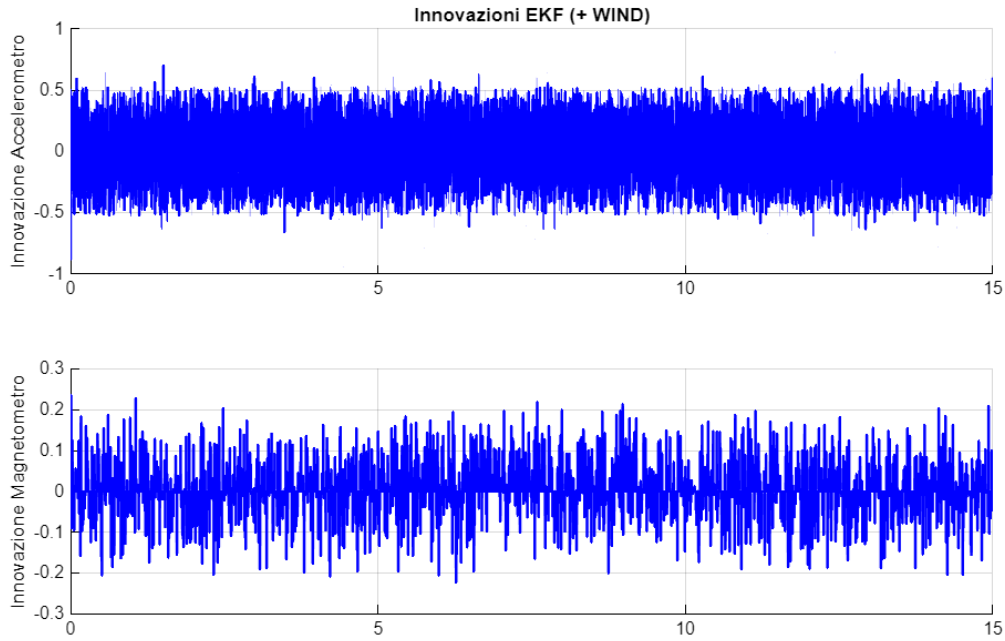


Figura 3.6: Innovazione EKF con la folata di vento.

L'impatto aerodinamico continuo a partire da $t = 5$ s genera una fisiologica discrepanza tra la dinamica reale (alterata dalla forza del vento) e il modello interno dell'osservatore (che ne assume l'assenza). Al momento dell'urto, le traiettorie angolari vere subiscono una deviazione, e l'EKF la insegue con un leggero scostamento transitorio.

Tuttavia, analizzando l'andamento degli errori di stima, emerge la robustezza dell'algoritmo. della nuova architettura implementata. A differenza di quanto accadrebbe con un filtro non ottimizzato, la perturbazione genera un'oscillazione dell'errore che **viene contenuta ampiamente all'interno dei limiti teorici di confidenza a $\pm 3\sigma$** , senza mai bucare i bound statistici.

Questo buon contenimento è confermato dal grafico dell'innovazione: contrariamente alle classiche derive che si registrano in presenza di bias esogeni non modellati, la sequenza dell'innovazione non presenta picchi severi, ma mantiene un profilo del tutto assimilabile a un rumore bianco stazionario. Grazie alla precisione dei Jacobiani analitici e alla rapidità della logica di correzione multi-rate a 1000 Hz, il filtro compensa efficacemente gli effetti del mismatch parametrico. e il gradino di vento in modo continuo e stabile.

4. Unscented Kalman Filter (UKF)

L'Unscented Kalman Filter (UKF) è una tecnica avanzata di stima stocastica progettata per ridurre gli errori dovuti alla linearizzazione dell'EKF in presenza di forti non-linearità. Invece di linearizzare analiticamente o numericamente il modello matematico attorno a un singolo punto operativo, l'UKF si basa sulla *Unscented Transform* (UT) [3]. Questo approccio deterministico seleziona un insieme minimo di punti campionati (denominati *Sigma Points*) posizionati attorno alla stima media corrente. Questi punti vengono poi propagati direttamente attraverso le equazioni non lineari originali dell'impianto, permettendo di catturare in modo accurato la media e la covarianza a posteriori senza ricorrere ad alcuna approssimazione al primo ordine.

4.1 Implementazione e Scelte Progettuali

Lo sviluppo dell'algoritmo UKF per il modello dell'elicottero 2DoF ha richiesto alcune specifiche scelte architetturali volte a ottimizzare il costo computazionale, l'accuratezza locale e la robustezza rispetto all'architettura multi-rate dei sensori.

La prima scelta architetturale rilevante riguarda la gestione del rumore di processo. Sebbene esista la formulazione *Augmented UKF* (che estende il vettore di stato per includere esplicitamente le variabili di rumore), tale approccio aumenta drasticamente le dimensioni del problema e il numero di Sigma Points da propagare. Assumendo invece che il rumore entri nel sistema in modo puramente additivo, nel presente progetto si è optato per una formulazione *Additive Noise UKF*. I punti sigma vengono generati esclusivamente sullo stato spaziale (4 dimensioni, generando $2n + 1 = 9$ Sigma Points) e la matrice di covarianza del rumore Q viene aggiunta a posteriori alla covarianza predetta. Questa soluzione abbassa sensibilmente l'onere computazionale garantendo prestazioni ottimali.

Inoltre, per adattare lo sparpagliamento dei Sigma Points alla dinamica specifica e al mismatch parametrico introdotto, i parametri di *scaling* della Unscented Transform sono stati ricalibrati rispetto alle formulazioni standard, impostando:

$$\alpha = 0.15, \quad \beta = 2, \quad \kappa = 0 \quad (4.1)$$

Tali valori definiscono l'ampiezza della distribuzione dei Sigma Points attorno alla stima media, bilanciando l'approssimazione locale delle non-linearità aerodinamiche con la stabilità numerica del filtro.

Infine, in stretta analogia con quanto implementato per l'EKF, la fase di correzione dell'UKF integra il *Gating* statistico tramite Distanza di Mahalanobis e la gestione condizionale dei sensori. In presenza di dati mancanti (es. magnetometro in attesa di campionamento), il filtro esclude temporaneamente le righe e le colonne non valide dalla covarianza dell'innovazione S e dalla covarianza incrociata P_{xy} .

Nel Listing 4.1 viene riportata l'implementazione completa utilizzata per la simulazione.

```

1 function [x_hat, P_corr, nu] = UKF_Filter(u_prev, y_k,
2     x_hat_prev, P_prev, Q, R, dt, p_hat)
3
4     % 1. Predizione UKF
5     [x_pred, P_pred] = UnscentedTransform_F(x_hat_prev,
6         P_prev, Q, dt, u_prev, p_hat);
7
8     % 2. Correzione UKF
9     [hat_y, S, Pxy] = UnscentedTransform_H(x_pred, P_pred, R,
10         p_hat);
11
12     % Logica Multi-rate e Gating Statistico
13     idx_validi = ~isnan(y_k);
14
15     x_hat = x_pred;
16     P_corr = P_pred;
17     nu = [0; 0; 0];
18
19     if any(idx_validi)
20         y_k_valid = y_k(idx_validi);
21         hat_y_valid = hat_y(idx_validi);
22
23         S_valid = S(idx_validi, idx_validi);
24         Pxy_valid = Pxy(:, idx_validi);
25
26         nu_valid = y_k_valid - hat_y_valid;
27         nu(idx_validi) = nu_valid;
28
29         % DISTANZA DI MAHALANOBIS
30         D2 = nu_valid' * (S_valid \ nu_valid);
31
32         gradi_liberta = sum(idx_validi);
33         if gradi_liberta == 1
34             soglia = 3.84;
35         else
36             soglia = 7.81;
37         end
38
39         if D2 < soglia
40             L = Pxy_valid / S_valid;
41             x_hat = x_pred + L * nu_valid;
42             P_corr = P_pred - L * S_valid * L';
43         end
44     end
45 end
46
47 % --- Funzioni Locali ---
48 function [mean_f, cov_f] = UnscentedTransform_F(x, P, Q, dt,
49     u, p_hat)

```

```

46     n = length(x);
47     alpha = 0.15; kappa = 0; beta = 2;
48     lambda = alpha^2 * (n + kappa) - n;
49
50     w_m = [lambda/(n+lambda), repmat(1/(2*(n+lambda)), 1,
51         2*n)];
52     w_c = w_m; w_c(1) = w_c(1) + (1 - alpha^2 + beta);
53
54     [U, S_mat, ~] = svd(P);
55     GAMMA = U * sqrt(S_mat);
56
57     sigmas = zeros(n, 2*n+1);
58     sigmas(:,1) = x;
59     for i = 1:n
60         sigmas(:, i+1) = x + sqrt(n+lambda)*GAMMA(:,i);
61         sigmas(:, i+1+n) = x - sqrt(n+lambda)*GAMMA(:,i);
62     end
63
64     sigmas_f = zeros(n, 2*n+1);
65     for i = 1:size(sigmas, 2)
66         al = sigmas(1,i); al_d = sigmas(2,i); be =
67             sigmas(3,i); be_d = sigmas(4,i);
68         J_b = p_hat.J_y * sin(al)^2 + (p_hat.J_z +
69             p_hat.m*p_hat.l^2)*cos(al)^2 + p_hat.I_b;
70
71         omega_body = [-be_d * sin(al); al_d; be_d * cos(al)];
72         v_body = cross(omega_body, [p_hat.l; 0; 0]);
73         v_rel_body = -v_body; % Il filtro assume vento
74             esterno nullo
75         v_rel_norm = norm(v_rel_body);
76
77         Cd_A = [0.11; 0.09; 0.22]; % Prodotto Cd * Area
78             perturbato
79
80         F_drag_body = 0.5 * p_hat.rho * Cd_A .* v_rel_body *
81             v_rel_norm;
82
83         tau_drag_alpha = -p_hat.l * F_drag_body(3);
84         tau_drag_beta = p_hat.l * F_drag_body(2);
85
86         al_dd = (-p_hat.c_alpha*al_d -
87             p_hat.m*p_hat.g*p_hat.l*sin(al) +
88             p_hat.l*u(1)*cos(be) +
89             p_hat.eps_p*p_hat.l*u(2)*sin(be) + tau_drag_alpha)
90             / p_hat.J_alpha;
91         be_dd = (-p_hat.c_beta*be_d + p_hat.l*u(2)*cos(al) +
92             p_hat.eps_y*p_hat.l*u(1)*sin(al) + tau_drag_beta)
93             / J_b;
94         sigmas_f(:,i) = [al + dt*al_d; al_d + dt*al_dd; be +
95             dt*be_d; be_d + dt*be_dd];
96     end

```

```

85     mean_f = sigmas_f * w_m';
86     tilde_f = sigmas_f - mean_f;
87     cov_f = (tilde_f .* w_c) * tilde_f' + Q;
88 end
89
90 function [mean_h, cov_h, cross_cov] = UnscentedTransform_H(x,
91     P, R, p_hat)
92     n = length(x);
93     alpha = 0.15; kappa = 0; beta = 2;
94     lambda = alpha^2 * (n + kappa) - n;
95
96     w_m = [lambda/(n+lambda), repmat(1/(2*(n+lambda)), 1,
97         2*n)];
98     w_c = w_m; w_c(1) = w_c(1) + (1 - alpha^2 + beta);
99
100     [U, S_mat, ~] = svd(P);
101     GAMMA = U * sqrt(S_mat);
102
103     sigmas = zeros(n, 2*n+1);
104     sigmas(:,1) = x;
105     for i = 1:n
106         sigmas(:, i+1) = x + sqrt(n+lambda)*GAMMA(:,i);
107         sigmas(:, i+1+n) = x - sqrt(n+lambda)*GAMMA(:,i);
108     end
109
110     sigmas_h = zeros(3, 2*n+1);
111     for i = 1:size(sigmas, 2)
112         sigmas_h(1,i) = -p_hat.g * sin(sigmas(1,i));
113         sigmas_h(2,i) = cos(sigmas(1,i)) * cos(sigmas(3,i));
114         sigmas_h(3,i) = -sin(sigmas(3,i));
115     end
116
117     mean_h = sigmas_h * w_m';
118     tilde_h = sigmas_h - mean_h;
119     cov_h = (tilde_h .* w_c) * tilde_h' + R;
120
121     tilde_x = sigmas - x;
122     cross_cov = (tilde_x .* w_c) * tilde_h';
123 end

```

Listing 4.1: Codice della funzione MATLAB per l'Unscented Kalman Filter (Additive Noise) con logica Multi-Rate.

4.2 Risultati della Stima UKF (Scenario Nominale)

Nello scenario in assenza di disturbi esogeni, ma operando sotto l'effetto dell'incertezza parametrica, l'UKF dimostra un tracciamento molto accurato della dinamica del sistema.

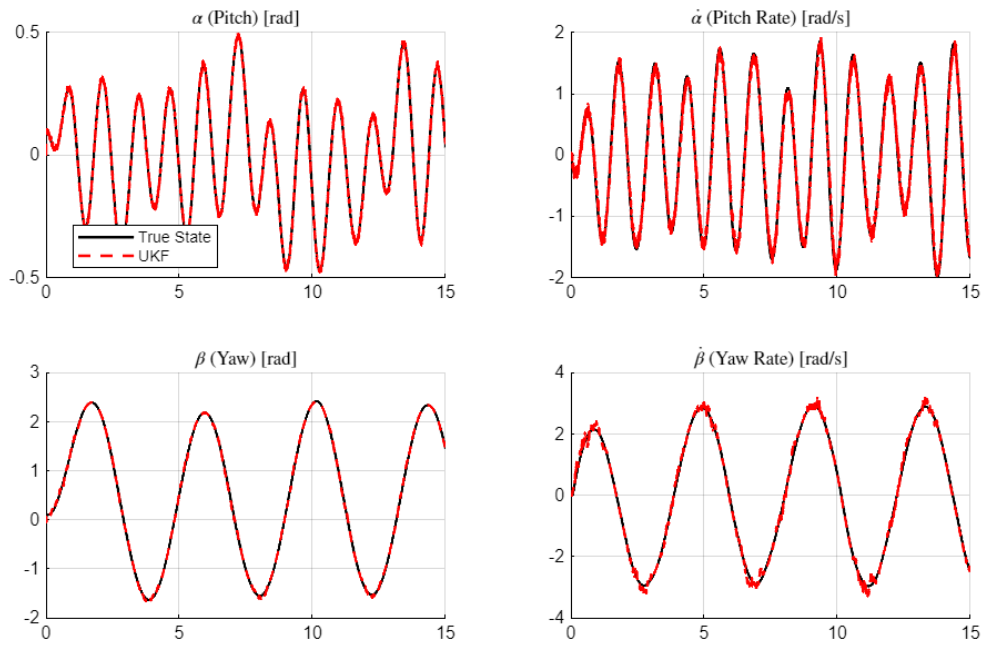


Figura 4.1: Tracciamento delle traiettorie angolari e delle velocità (UKF vs Ground Truth) in assenza di vento.

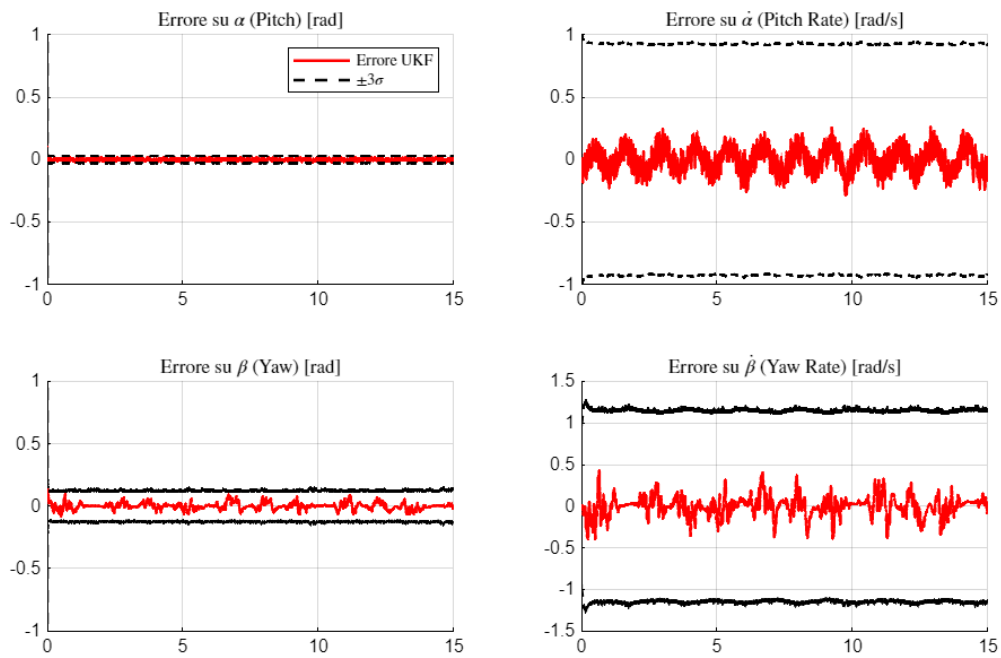


Figura 4.2: Errori UKF a $\pm 3\sigma$ in assenza di vento.

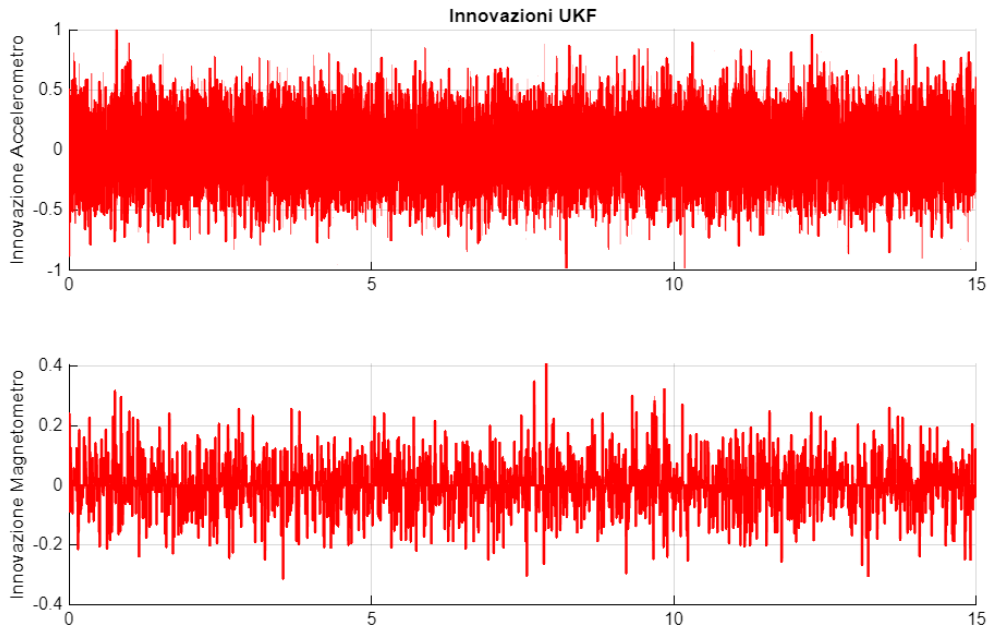


Figura 4.3: Innovazione UKF in assenza di vento.

Confrontando visivamente l'errore di stima e l'andamento dei limiti di confidenza, le performance dell'UKF risultano quasi indistinguibili da quelle dell'EKF. L'errore si mantiene a media nulla all'interno dei bordi statistici e la sequenza dell'innovazione appare stazionaria. Questo fenomeno è imputabile all'elevata frequenza di campionamento (1000 Hz, $dt = 0.001$ s) e all'assenza di scostamenti repentini, che impediscono alle forti non-linearità del modello di divergere in modo drastico. Abbiamo osservato, il passaggio positivo del test di Anderson ci dice che l'innovazione risulta essere un rumore bianco per tutti i sensori attivi.

4.3 Risultati della Stima UKF con Disturbo Aerodinamico

Sotto l'impatto del gradino continuo di vento a $t = 5$ s, l'UKF conferma la robustezza già osservata per l'algoritmo al primo ordine, gestendo con estrema efficacia la combinazione tra incertezza di modello e disturbo esogeno.

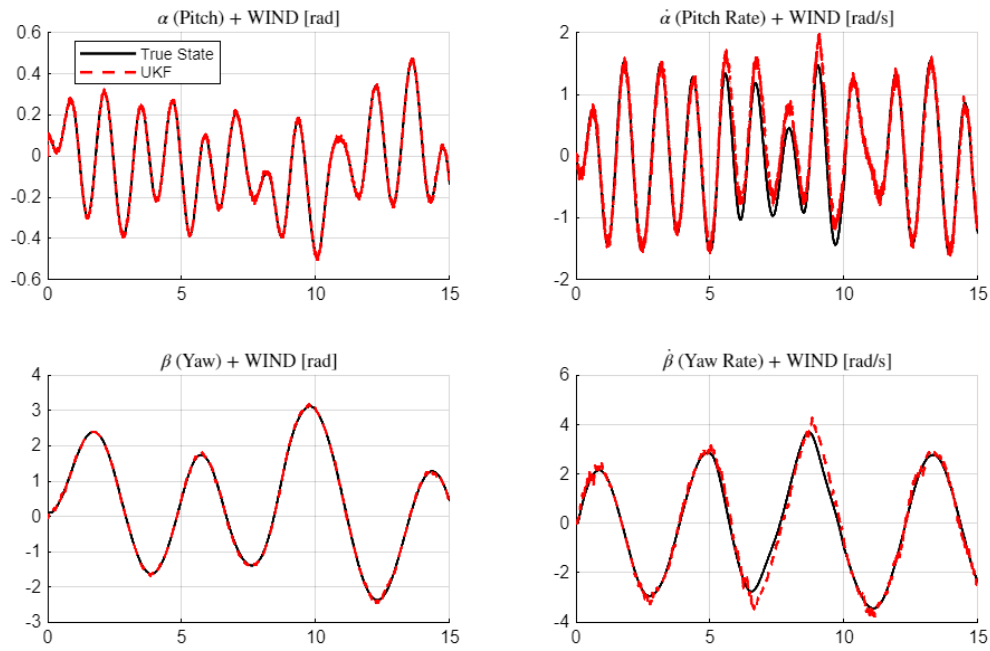


Figura 4.4: Tracciamento delle traiettorie angolari e delle velocità (UKF vs Ground Truth) con la folata di vento.

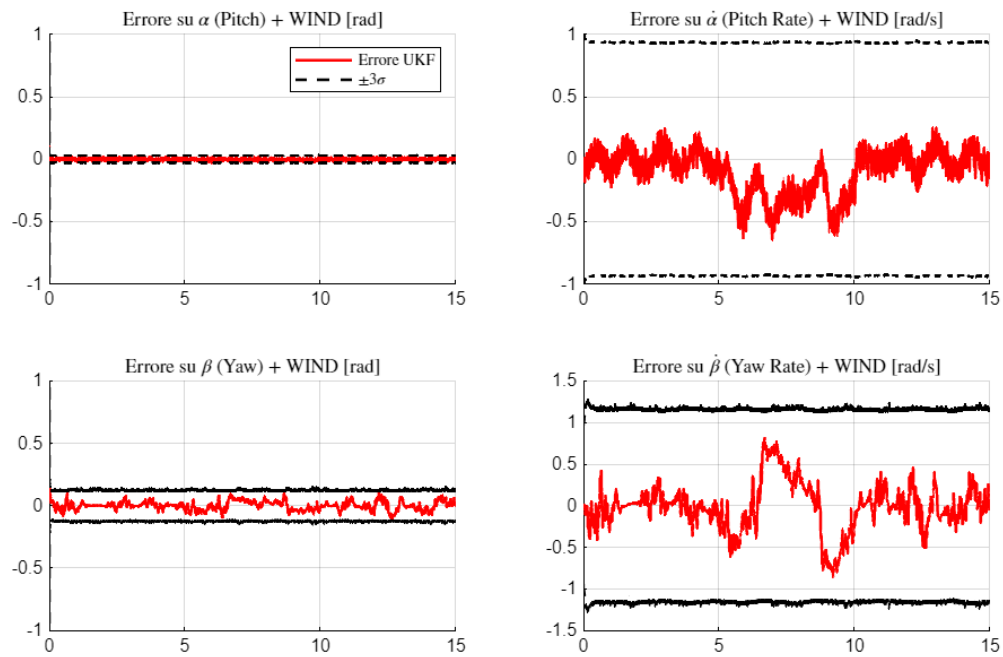


Figura 4.5: Errori UKF a $\pm 3\sigma$ con la folata di vento.

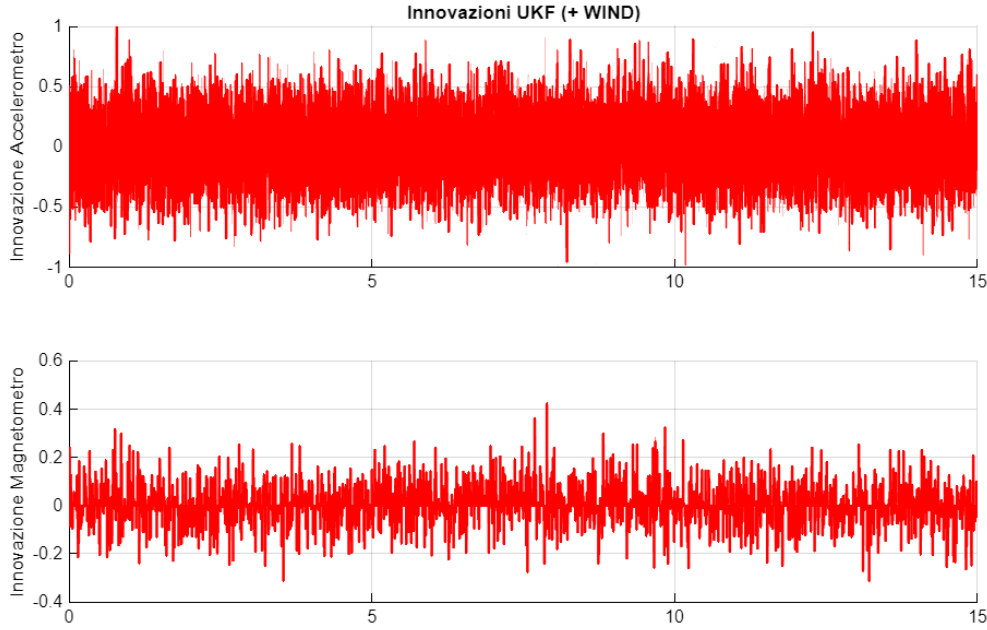


Figura 4.6: Innovazione UKF con la folata di vento.

Il disturbo aerodinamico continuo genera una fisiologica alterazione della dinamica del sistema, ma l'analisi degli errori evidenzia l'assoluta stabilità dell'osservatore. Contrariamente a quanto accadeva con implementazioni meno affinate, la propagazione non lineare dei Sigma Points e il filtraggio di Mahalanobis contribuiscono a garantire che l'errore di stima rimanga sempre ben confinato all'interno dei limiti teorici di confidenza a $\pm 3\sigma$, senza registrare alcuna violazione statistica. Anche l'innovazione non presenta più derive o picchi severi al momento dell'impatto, preservando un andamento stazionario del tutto assimilabile a un rumore bianco. Il filtro assorbe la perturbazione garantendo un tracking reattivo e robusto, del tutto equiparabile a quello dell'EKF ottimizzato.

4.4 Analisi Quantitativa e Confronto RMSE (EKF vs UKF)

Per sancire quale stimatore online offra le prestazioni migliori, è stata condotta un'analisi quantitativa globale valutando l'Errore Quadratico Medio (RMSE - *Root Mean Square Error*) per tutte le variabili di stato in entrambi gli scenari.

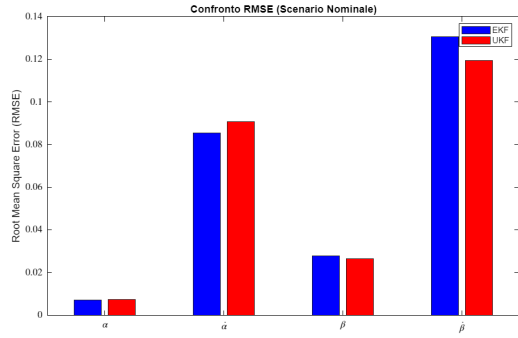


Figura 4.7: Confronto RMSE in condizioni nominali.

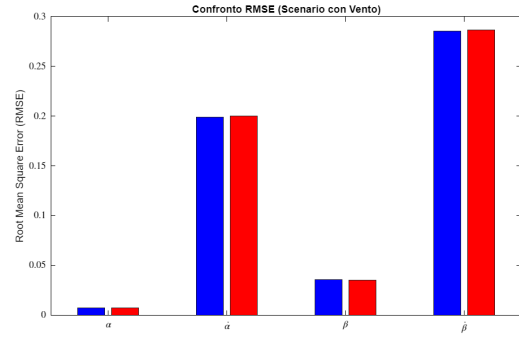


Figura 4.8: Confronto RMSE con disturbo aerodinamico.

L'analisi dei grafici a barre evidenzia un risultato ingegneristicamente rilevante che sottolinea la validità teorica dell'UKF. Nello scenario nominale, si osserva un comportamento misto: l'EKF mantiene un Errore Quadratico Medio marginalmente inferiore sulla stima della derivata di pitch ($\dot{\alpha}$), ma l'UKF riesce a sovraperformare l'algoritmo al primo ordine sulla stima della derivata di yaw ($\dot{\beta}$). Sulle variabili di posizione angolare (α e β), le performance dei due stimatori risultano pressoché identiche. Questo dimostra che la dispersione spaziale dei Sigma Points dell'UKF riesce a catturare dinamiche complesse e non lineari (come l'inerzia variabile sull'asse di imbardata) in modo leggermente più fedele rispetto all'approssimazione Jacobiana al primo ordine.

Nel caso dell'esperimento condotto con il vento continuo, si nota un fisiologico innalzamento generale degli indici di errore dovuto al transitorio iniziale e all'alterazione dinamica permanente introdotta dalla forzante aerodinamica. In questa severa condizione di stress, le performance globali dei due stimatori si allineano perfettamente, dimostrando la medesima capacità di reiezione del bias.

Si conclude che per questo specifico 2DoF Helicopter, l'uso dell'EKF risulta spesso la scelta più giusta a fini pratici, garantendo buone prestazioni a fronte di un costo computazionale sensibilmente ridotto grazie all'impiego dei Jacobiani analitici. Tuttavia, l'UKF dimostra concretamente il suo valore: non solo compensa efficacemente gli effetti dei disturbi esogeni, ma riesce persino a ottimizzare lievemente la stima su specifiche dinamiche fortemente non lineari del modello (come lo yaw rate), validando e rafforzando la qualità dell'intera architettura di stima sviluppata.

5. Smoothing Offline: Algoritmo Rauch-Tung-Striebel (RTS)

L'Extended Kalman Filter (EKF) analizzato nei capitoli precedenti è uno stimatore *causale*: ad ogni istante k , la stima dello stato si basa esclusivamente sulle misurazioni acquisite fino a quell'istante. Questa caratteristica lo rende lo standard industriale per le applicazioni in tempo reale. Tuttavia, in fase di post-processing (analisi offline dei dati), è possibile sfruttare anche le misurazioni future per migliorare l'accuratezza della stima passata.

Lo Smoother di Rauch-Tung-Striebel (RTS) è un algoritmo di stima ottimale non causale che opera in due fasi distinte (Two-Pass Smoother) [5]:

1. **Forward Pass (EKF)**: Il filtro EKF classico viene eseguito dal primo all'ultimo istante di simulazione ($k = 1 \dots N$). Durante questa fase, vengono salvati in memoria lo stato stimato $\hat{x}_{k|k}$, lo stato predetto $\hat{x}_{k+1|k}$, la covarianza corretta $P_{k|k}$, la covarianza predetta $P_{k+1|k}$ e la matrice Jacobiana analitica della dinamica F_k .
2. **Backward Pass (RTS)**: A simulazione conclusa, l'algoritmo riparte dall'ultimo istante ($k = N$) e procede a ritroso verso $k = 1$. In questa fase, l'algoritmo calcola uno stato "smussato" (*smoothed*) aggiornando le stime dell'andata con le misure degli istanti futuri.

Le equazioni ricorsive del backward pass per il calcolo dello stato smoothed $\hat{x}_{k|N}$ e della covarianza smoothed $P_{k|N}$ sono:

$$C_k = P_{k|k} F_k^T P_{k+1|k}^{-1} \quad (5.1)$$

$$\hat{x}_{k|N} = \hat{x}_{k|k} + C_k (\hat{x}_{k+1|N} - \hat{x}_{k+1|k}) \quad (5.2)$$

$$P_{k|N} = P_{k|k} + C_k (P_{k+1|N} - P_{k+1|k}) C_k^T \quad (5.3)$$

dove C_k rappresenta il Guadagno dello Smoother.

5.1 Implementazione e Scelte Progettuali

L'implementazione software dello Smoother RTS ha richiesto l'estrazione completa del *data-log* dal modello Simulink per l'elaborazione *offline* in ambiente MATLAB. Tale registro di memoria contiene l'intero set di risultati della prima passata (*forward pass*), registrati alla frequenza di 1000 Hz ($dt = 0.001$ s). Questi includono le stime degli stati, le covarianze (aggiornate con logica multi-rate) e le matrici Jacobiane esatte calcolate in tempo reale dall'EKF, elementi strettamente necessari per abilitare la successiva ricorsione all'indietro.

Di seguito si riporta lo script MATLAB sviluppato per l'esecuzione del calcolo matriciale *backward* a partire dai dati estratti da Simulink.

```

1 % Esecuzione dello Smoother RTS offline sui dati estratti da Simulink
2 clear all; close all; clc;
3 disp('Caricamento dei dati da simulink_smoother_data.mat...');
4 load('simulink_smoother_data.mat'); % Carica t, N, x_T, log_EKF (dt =
   0.001s)
5
6 % =====
7 % 1. SMOOTHER DI RAUCH-TUNG-STRIEBEL (RTS) OFFLINE
8 % =====
9 disp('Esecuzione dello Smoother RTS...');
10 log_EKF.x_hat_smoothed = zeros(4, N);
11 log_EKF.P_smoothed = zeros(4, 4, N);
12
13 % Inizializzazione all'ultimo istante (t = N)
14 log_EKF.x_hat_smoothed(:, N) = log_EKF.x_hat(:, N);
15 log_EKF.P_smoothed(:, :, N) = log_EKF.P_corr(:, :, N);
16
17 % =====
18 % RICORSIONE BACKWARD
19 % =====
20 % Matrice di stabilizzazione backward
21 Q_RTS = diag([1e-2, 1e-2, 1e-2, 1e-2]);
22
23 for k = N-1:-1:1
24     P_k_k = log_EKF.P_corr(:, :, k);
25     P_k1_k = log_EKF.P_pred(:, :, k+1);
26     F_k = log_EKF.F_matrix(:, :, k+1);
27
28     % Aggiunta di Q_RTS per stabilizzazione robusta (previene
       esplosioni/divisione per zero)
29     P_k1_k_robusto = P_k1_k + Q_RTS;
30
31     % Calcolo robusto del guadagno C_k
32     C_k = P_k_k * F_k' / P_k1_k_robusto;
33
34     % Aggiornamento Stato
35     err_x = log_EKF.x_hat_smoothed(:, k+1) - log_EKF.x_hat_pred(:, k+1);
36     log_EKF.x_hat_smoothed(:, k) = log_EKF.x_hat(:, k) + C_k * err_x;
37
38     % Aggiornamento Covarianza e Forzatura della Simmetria
39     err_P = log_EKF.P_smoothed(:, :, k+1) - P_k1_k;
40     P_smooth_temp = P_k_k + C_k * err_P * C_k';
41     log_EKF.P_smoothed(:, :, k) = (P_smooth_temp + P_smooth_temp') / 2;
42 end
43
44 % Calcolo Errori per confronto
45 err_EKF = x_T - log_EKF.x_hat;
46 err_RTS = x_T - log_EKF.x_hat_smoothed;
47
48 % [Nota: l'implementazione iterativa estesa per le casistiche di vento
   segue
49 % la medesima architettura algoritmica applicata alle strutture
   log_EKF.*_W]

```

Listing 5.1: Algoritmo RTS Smoother Offline eseguito sul datalog dell'EKF multi-rate.

5.2 Analisi dei Risultati in Condizioni Nominali (Assenza di Vento)

La prima campagna di validazione dello Smoother RTS è stata condotta in assenza di disturbi esogeni esterni, ma in presenza del forte mismatch parametrico del $\pm 15\%$ illustrato in precedenza, confrontando le prestazioni offline con quelle real-time dell'EKF.

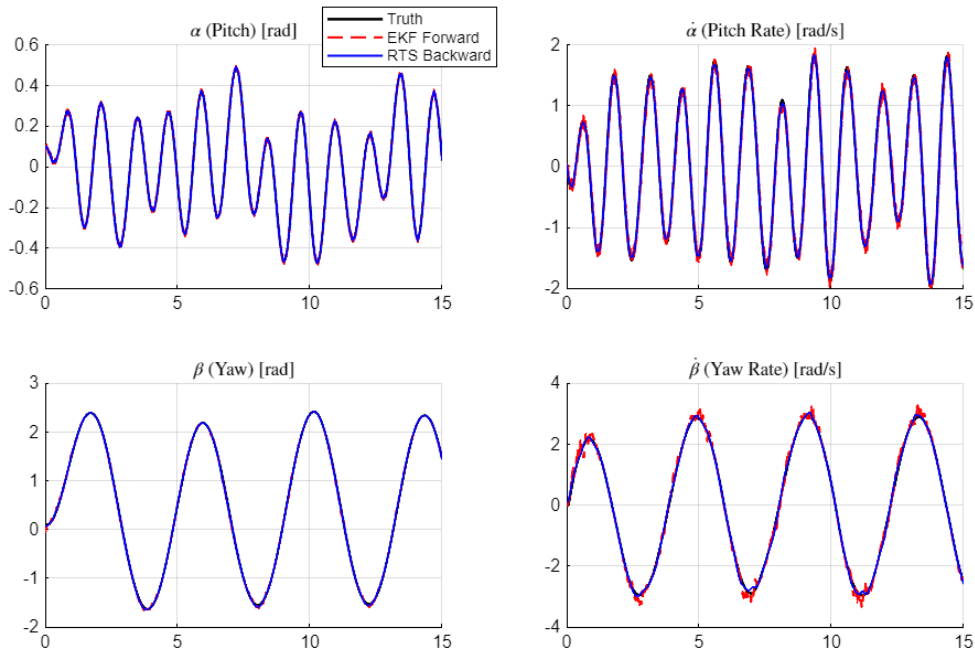


Figura 5.1: Tracciamento delle traiettorie angolari e delle velocità (EKF vs RTS) in condizioni nominali con mismatch parametrico.

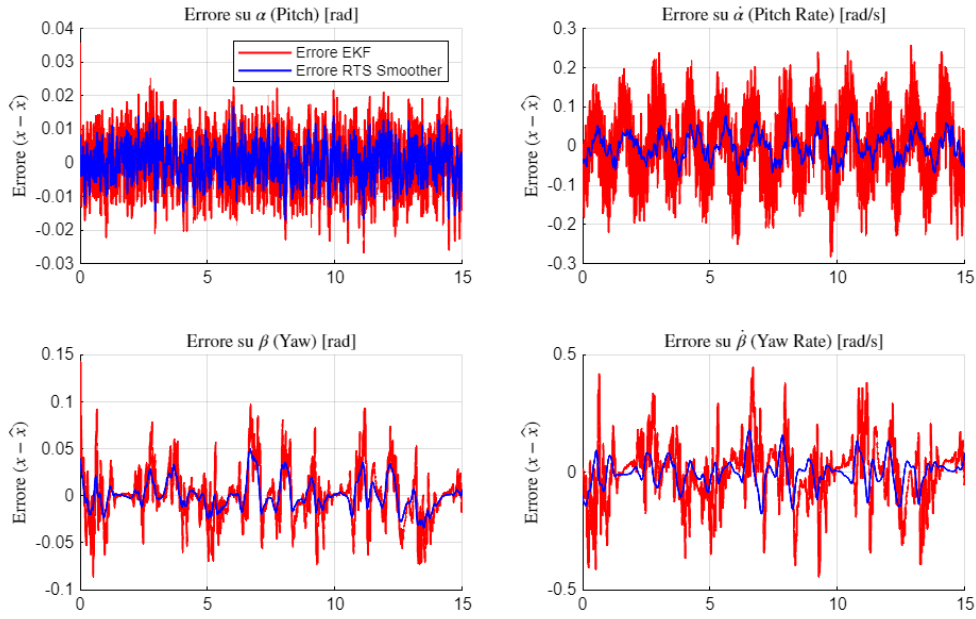


Figura 5.2: Errore di stima (EKF vs RTS) in assenza di vento.

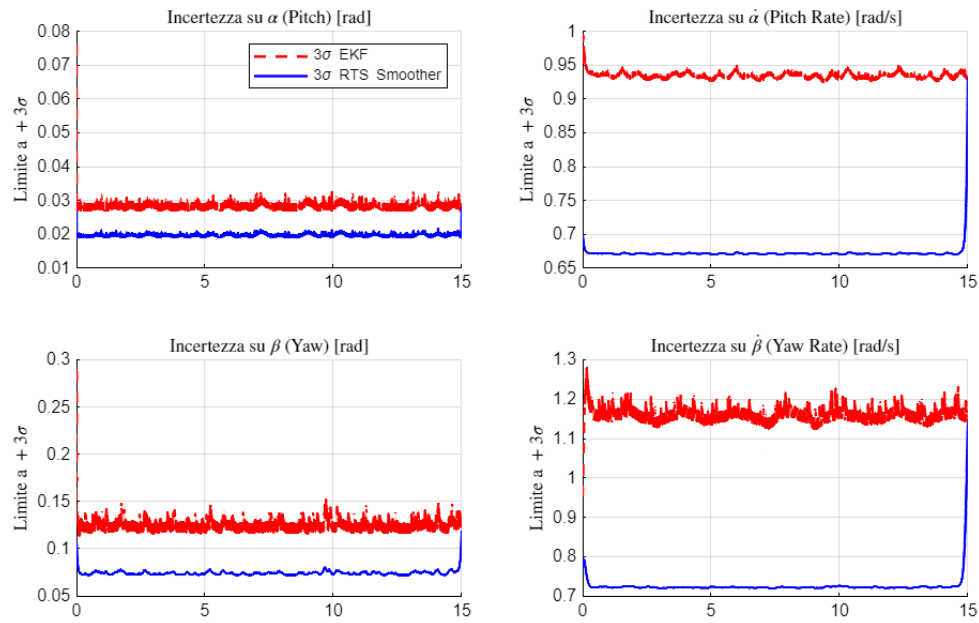


Figura 5.3: Incertezza a $\pm 3\sigma$ (EKF vs RTS) in condizioni nominali.

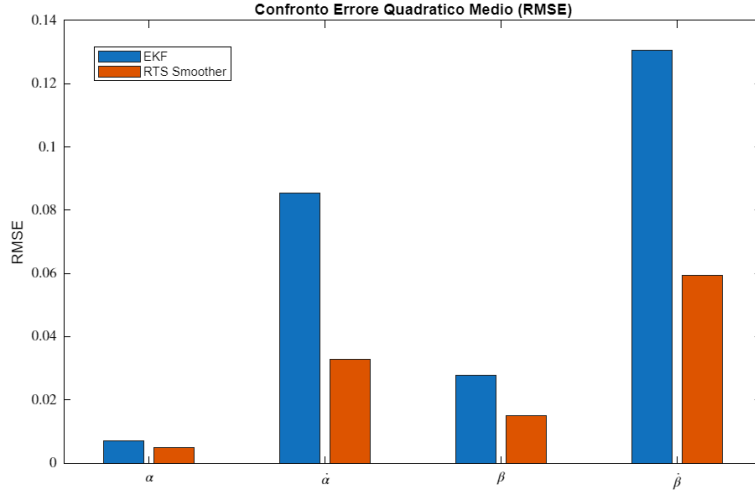


Figura 5.4: Confronto Errore Quadratico Medio (RMSE) tra EKF e RTS in assenza di vento.

Se dall'osservazione macroscopica delle traiettorie (Figura 5.1) le stime causali e non causali appaiono pressoché sovrapposte, un'analisi di dettaglio rivela il profondo impatto dell'algoritmo RTS. Valutando l'andamento dell'errore di stima (Figura 5.2), emerge con chiarezza che il *backward pass* riesce a filtrare e smussare gran parte del rumore stocastico residuo tipico del filtro *forward*, confinando l'errore in una banda di oscillazione nettamente più ristretta e pulita.

Questo formidabile miglioramento qualitativo è quantificato in modo inequivocabile dal grafico dell'RMSE (Figura 5.4). Lo Smoother RTS abbatte l'Errore Quadratico Medio su tutte le variabili di stato. L'intervento è particolarmente incisivo sulle derivate di assetto: l'RMSE del *Pitch Rate* ($\dot{\alpha}$) e dello *Yaw Rate* ($\dot{\beta}$) viene più che dimezzato rispetto alle prestazioni, pur eccellenti, del filtro EKF causale.

A riprova di questo consolidamento, i limiti di confidenza statistica a $\pm 3\sigma$ (Figura 5.3) subiscono un netto restringimento. L'incertezza globale della stima *offline* collassa a valori marginali (nell'ordine di 0.02 rad per le posizioni e 0.65 rad/s per le velocità), a dimostrazione di come lo sfruttamento delle misure future consenta all'RTS di conferire al sistema una stabilità assoluta.

5.3 Analisi dei Risultati con Disturbo Aerodinamico (Vento Continuo)

Per testare i limiti operativi dell'algoritmo *offline*, è stato analizzato il secondo scenario, caratterizzato dal gradino continuo di vento a 1 m/s a partire da $t = 5$ s, persistendo nel mismatch parametrico.

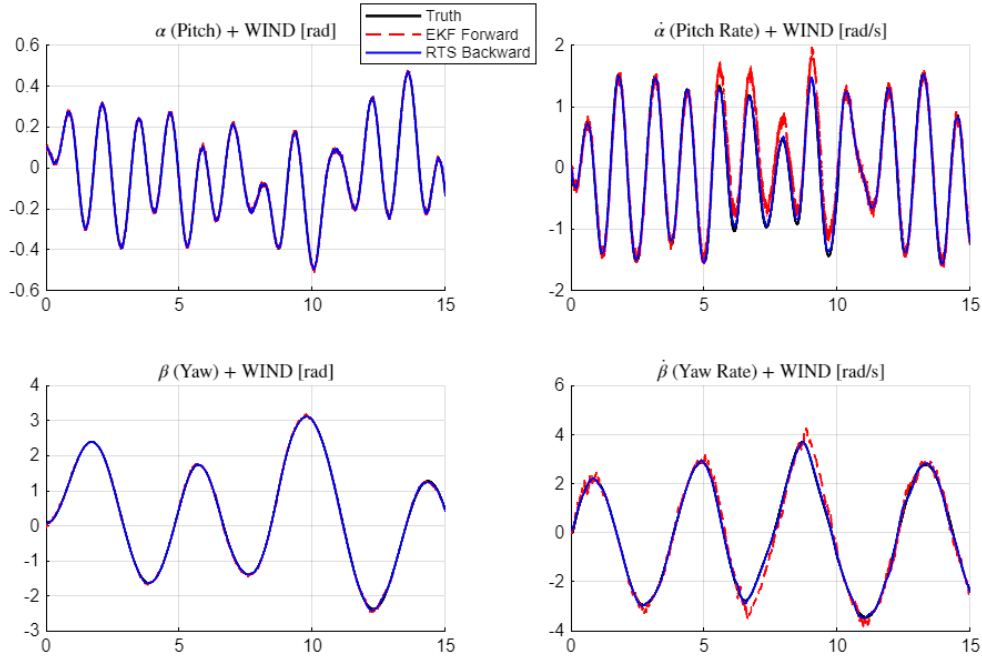


Figura 5.5: Tracciamento delle traiettorie (EKF vs RTS) in presenza di gradino di vento continuo da $t = 5s$.

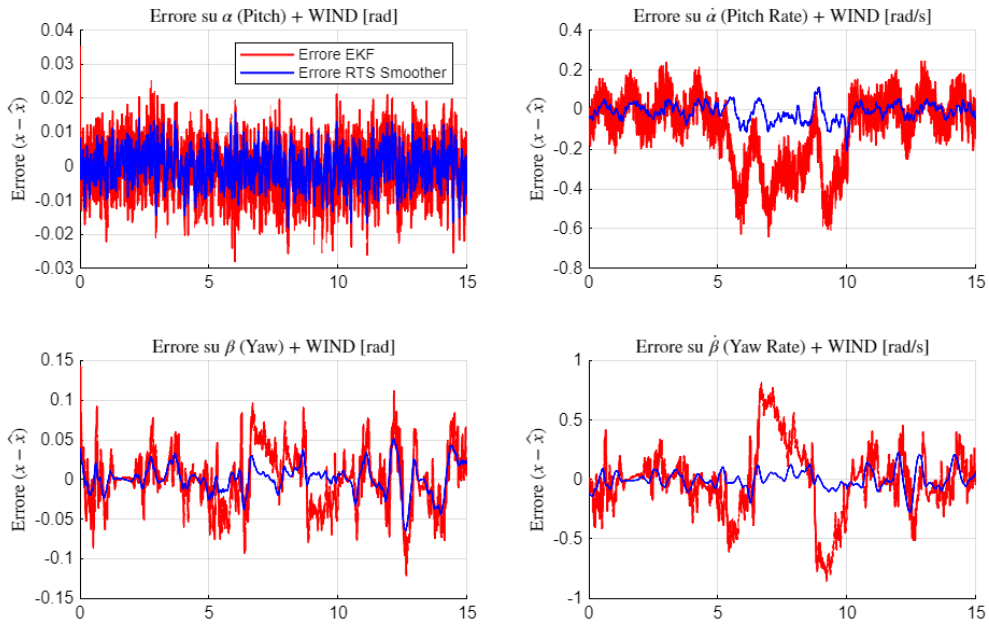


Figura 5.6: Errore di stima (EKF vs RTS) in presenza di gradino di vento.

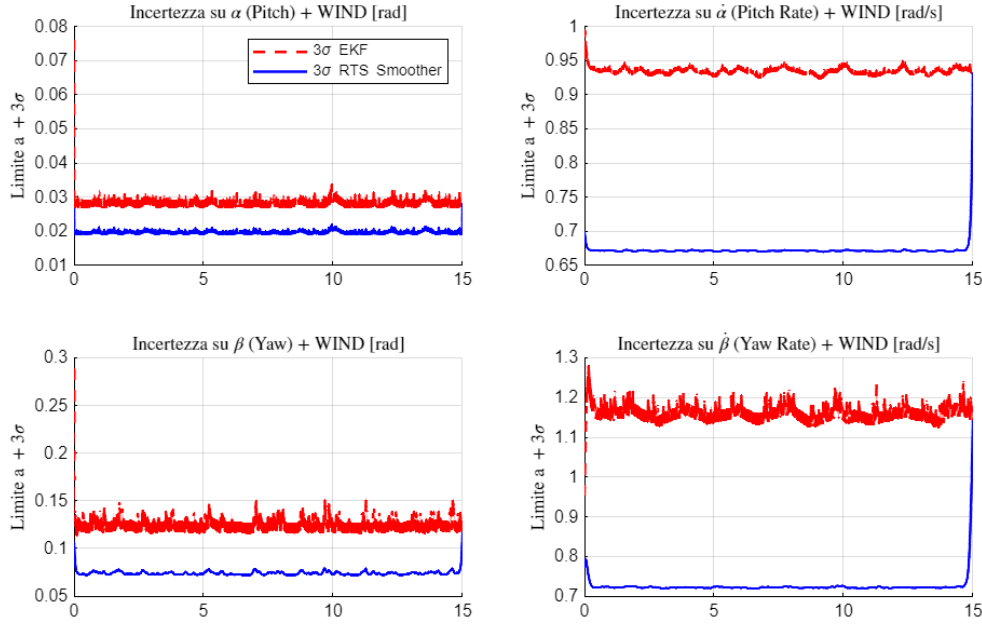


Figura 5.7: Incertezza a $\pm 3\sigma$ (EKF vs RTS) in presenza di vento continuo.

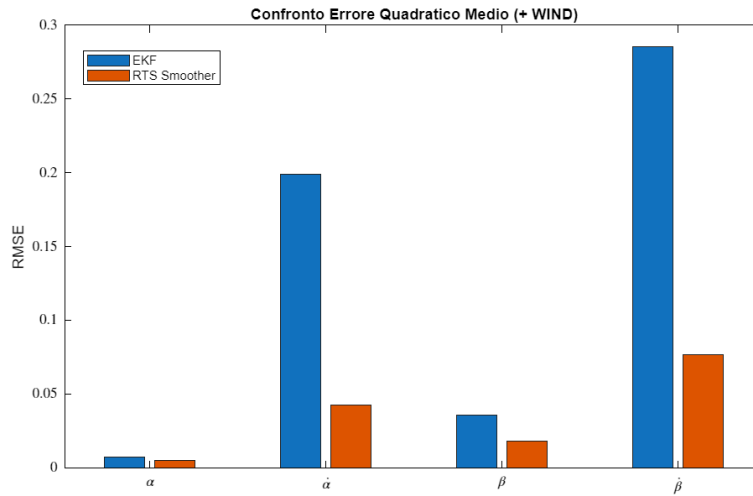


Figura 5.8: Confronto Errore Quadratico Medio (RMSE) tra EKF e RTS con vento continuo.

L'introduzione della forzante aerodinamica prolungata rappresenta una sfida severa. Come visibile in Figura 5.6, l'EKF subisce l'impatto deviando temporaneamente, per poi inseguire il nuovo regime dinamico con una dispersione inevitabilmente aumentata rispetto allo scenario nominale. Tuttavia, lo Smoother RTS dimostra una capacità correttiva straordinaria. Sfruttando le misurazioni raccolte a valle dell'evento, la ricorsione *backward* "prevede" l'impatto aerodinamico: l'errore dell'RTS risulta enormemente attenuato, mantenendo un tracciamento fluido e respingendo la componente rumorosa amplificata dal vento.

L'entità di questo vantaggio è sancita definitivamente dal grafico comparativo dell'RMSE (Figura 5.8). Lo Smoother riesce ad abbattere drasticamente l'errore globale introdotto dal disturbo esogeno non modellato: l'RMSE del *Pitch Rate* crolla da circa 0.20 a meno di 0.05, mentre quello dello *Yaw Rate* precipita da 0.28 a poco più di 0.07.

Questi dati ribaltano la concezione per cui un filtro non lineare non possa mitigare efficacemente bias deterministici prolungati. Conoscendo in anticipo l'evoluzione della traiettoria alterata dal vento, il *backward pass* riallinea l'intera stima passata verso il *Ground Truth*, compensando con formidabile efficacia le lacune del modello e restituendo un risultato *offline* qualitativamente e quantitativamente superiore.

6. Sviluppi futuri

Alla luce dei risultati ottenuti, i futuri sviluppi di questo progetto potrebbero orientarsi verso l'implementazione di un Filtro di Kalman a Stato Aumentato (*Augmented State EKF*) [2, 1]. Includendo le forze esogene direttamente nel vettore di stato, il sistema non si limiterebbe a subire e compensare passivamente il disturbo aerodinamico continuo, ma sarebbe in grado di stimare attivamente in tempo reale l'intensità e la direzione del vento, aprendo la strada a logiche di controllo predittivo e reiezione attiva delle turbolenze.

Un ulteriore avanzamento di grande interesse ingegneristico consiste nell'adozione di un Filtro di Kalman Adattativo (AKF). Nell'attuale architettura, pur avendo implementato con successo una logica di *Gating* statistico (Mahalanobis) per il filtraggio delle anomalie, le matrici di covarianza del rumore di processo (Q) e di misura (R) sono state calibrate *offline* tramite un *tuning* statico di compromesso. Tale bilanciamento è stato studiato per gestire contemporaneamente l'incertezza dovuta al mismatch parametrico e l'impatto del gradino di vento. Un approccio adattativo, basato su tecniche di *covariance matching* [4], consentirebbe al filtro di analizzare statisticamente l'innovazione residua e di ricalibrare dinamicamente le matrici Q e R in tempo reale. Ciò conferirebbe allo stimatore la capacità di adattare autonomamente il proprio grado di "fiducia" nel modello interno o nei sensori a seconda delle variazioni ambientali, massimizzando l'ottimalità della stima in scenari operativi altamente mutevoli e asincroni.

7. Conclusioni

Il presente elaborato ha affrontato con successo il problema della stima stocastica dello stato per un sistema fortemente non lineare e accoppiato, modellato sulle dinamiche di un Elicottero a due gradi di libertà (2DoF). L'obiettivo primario di ricostruire l'assetto e le velocità angolari a partire da misurazioni indirette, rumorose e asincrone (architettura sensoriale multi-rate) è stato raggiunto progettando, implementando e confrontando tre diverse architetture di filtraggio: l'Extended Kalman Filter (EKF), l'Unscented Kalman Filter (UKF) e lo Smoother offline di Rauch-Tung-Striebel (RTS). L'intero impianto di stima è stato validato in scenari operativi severi, caratterizzati da un forte mismatch parametrico e da disturbi esogeni prolungati.

L'analisi comparativa in tempo reale tra EKF e UKF ha restituito risultati ingegneristicamente rilevanti. Grazie all'impiego dei Jacobiani analitici esatti e a una logica di integrazione ad altissima frequenza (1000 Hz), l'EKF ha dimostrato una stabilità eccezionale, confermandosi una soluzione formidabile in termini di rapporto tra onere computazionale e accuratezza. Tuttavia, l'UKF ha mostrato concretamente la sua superiorità teorica nella gestione delle dinamiche più complesse: ha infatti sovraperformato l'algoritmo al primo ordine nella stima della derivata di imbardata (*Yaw Rate*), dimostrando che la propagazione spaziale dei *Sigma Points* cattura con maggiore fedeltà le variazioni intrinseche dell'inerzia. Entrambi gli algoritmi, potenziati dal filtraggio statistico di Mahalanobis (*Gating*), hanno garantito il mantenimento degli errori costantemente all'interno dei limiti teorici a $\pm 3\sigma$, annullando le derive dell'innovazione.

L'applicazione dello Smoother RTS in fase di *post-processing* ha fornito le evidenze più d'impatto sulla natura della stima ottima. L'implementazione dell'algoritmo *backward* ha richiesto la risoluzione di criticità complesse legate all'inversione matriciale e all'instabilità della dinamica retrograda (anti-attrito), superate tramite l'introduzione di una matrice di regolarizzazione additiva (Q_{RTS}). I risultati ottenuti hanno dimostrato che lo *smoothing offline* apporta un miglioramento drastico alla qualità della stima: sfruttando le misure future, l'RTS ha dimezzato l'Errore Quadratico Medio (RMSE) nominale sulle velocità rispetto all'eccellente filtro causale *forward* e ha abbattuto drasticamente l'errore globale in presenza di disturbo. Questo certifica l'immensa potenza dell'approccio non causale nell'estrarre informazioni latenti e nell'annullare il rumore stocastico residuo.

Infine, l'iniezione del disturbo aerodinamico continuo (gradino di vento) ha testato i limiti degli algoritmi in condizioni prolungate di *model mismatch*. Tutti gli stimatori hanno dimostrato una straordinaria capacità di reiezione del disturbo: l'integrazione del Guadagno di Kalman ha permesso di assorbire l'impatto e gestire la deviazione dinamica permanente, senza mai causare la divergenza statistica dei filtri. Lo Smoother RTS, in particolare, ha mitigato con formidabile efficacia il *bias* deterministico esogeno, riallineando l'intera stima passata verso il *Ground*

Truth e dimostrando la validità assoluta e la solidità dell'intera architettura di stima sviluppata.

Bibliografia

- [1] Bernard Friedland. Treatment of bias in recursive filtering. *IEEE Transactions on Automatic Control*, 14(4):359–367, 1969.
- [2] Mohinder S Grewal and Angus P Andrews. *Kalman filtering: theory and practice using MATLAB*. John Wiley & Sons, 2014.
- [3] Simon J Julier and Jeffrey K Uhlmann. New extension of the kalman filter to nonlinear systems. In *Signal processing, sensor fusion, and target recognition VI*, volume 3068, pages 182–193. SPIE, 1997.
- [4] Raman K Mehra. On the identification of variances and adaptive kalman filtering. *IEEE Transactions on automatic control*, 15(2):175–184, 1970.
- [5] Herbert E Rauch, F Tung, and CT Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA journal*, 3(8):1445–1450, 1965.
- [6] Dan Simon. *Optimal state estimation: Kalman, H infinity, and nonlinear approaches*. John Wiley & Sons, 2006.
- [7] A. Tastemirov, A. Lecchini-Visintini, and R. M. Morales-Viviescas. Complete dynamic model of the twin rotor mimo system (trms) with experimental validation. *Control Engineering Practice*, 66:89–98, 2017.
- [8] Eric A. Wan and Rudolph Van Der Merwe. The unscented kalman filter for nonlinear estimation. In *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (AS-SPCC)*, pages 153–158, 2000.